

Desarrollo de software en sistemas distribuidos

Resumen teorías

Clase 1 - Sistemas distribuidos	7
Evolución de los Sistemas de Información	7
Arquitectura Monolítica	7
Arquitectura Distribuida (2 capas y N capas)	8
2 capas	8
Cliente/servidor	8
N capas	10
Arquitecturas Distribuidas con objetos (estándar CORBA)	10
Integración de aplicaciones	10
Integración punto a punto	10
Por adaptadores	11
Por bus o mediador	11
Arquitecturas Orientadas a Servicios	12
Definiciones de sistemas distribuidos	12
Middleware	12
Componente middleware	14
Modelos de middleware	14
Middleware por socket	14
Middleware por RPC o RMI	15
Middleware orientado a mensajes	15
MOM vs RPC	16
Arquitectura distribuida	16
El protocolo de transporte del middleware	17
Servicios de seguridad distribuidos	17
Nivel C2 de seguridad	18
Clase 2 - Gestión de proyecto	19
Clase 3 - Procesos de negocio	20
Orientación de una organización a procesos de negocio	20
Beneficios obtenidos para la organización	20
Beneficios obtenidos para el analista	21
BPM	21
Proceso	21
Elementos	21
Mapa y diagrama de procesos	22
Elementos básicos de un BPD (BP Diagram)	22
Actores y propietarios	23

Reglas de negocio	23
Ciclo de vida clásico	24
Ciclo de vida según M. Weske	25
BPMS	25
Componentes tecnológicos de los BPMS	25
Aporte de los BPMS	26
BAM	26
Clase 4 - Servicios web	27
Evolución de las aplicaciones	27
Arquitectura basada en servicios SOA	27
Características de SOA	27
Principios de diseño SOA	28
Actores SOA	29
Servicios web	29
Características de los servicios	29
SOAP vs REST	30
SOAP	30
Requisitos	31
Estructura de un mensaje SOAP	33
Pila de protocolos	34
Estructura de un documento WSDL	34
El protocolo UDDI	34
RESTful	35
Reglas de diseño	36
GraphQL	36
Conceptos previos	36
Problemas de REST	37
Ventajas de GraphQL	37
Desventajas de GraphQL	37
REST vs GraphQL	38
Principios de diseño	38
Conclusiones	39
Clase 5 - Cloud computing	40
Datacenters	41
Virtualización	41
Múltiples servidores físicos	41
Múltiples servidores virtuales	42
Cloud computing	43
Modelo 5-3-4	44
5 características esenciales	44
On-demand self-service	44
Broad network access	45
Resource pooling	45
Rapid elasticity	45

Measured Service	45
3 modelos de servicio	45
IaaS	45
Ejemplos	46
Funciones de gestión	46
PaaS	47
Ejemplos	47
Tipos de ofertas	47
¿Qué provee?	48
SaaS	48
Ejemplos	49
Beneficios	49
Desventajas	49
Beneficios SaaS vs on-premise	50
SaaS vs ASP (Application Service Provider)	50
Variantes aaS	51
4 modelos de despliegue	51
Cloud público	51
Cloud privado	52
Cloud híbrido	53
Cloud comunitario	54
Otros modelos de despliegue	54
Resumen	55
Estándares en el cloud	55
Estándares IaaS	56
OVF (Open Virtualization Format)	56
CIMI (Cloud Infrastructure Management Interface)	56
OCCI (Open Cloud Computing Interface)	56
CDMI (Cloud Data Management Interface)	57
Estándares PaaS	57
CAMP (Cloud Application Management for Platforms)	57
Seguridad en cloud computing	58
Infraestructura de seguridad a nivel de red	58
Asegurar la confidencialidad e integridad de los datos transmitidos desde y hacia el proveedor de cloud	58
Asegurar un protocolo de control de acceso propio	59
Asegurar la disponibilidad de los recursos expuestos a Internet	59
Reemplazar el modelo establecido de zonas, niveles y dominios	60
¿Cómo podemos mitigar los factores de riesgo?	60
Infraestructura de seguridad a nivel de host	61
Seguridad a nivel de host en SaaS y PaaS	61
Seguridad a nivel de host en IaaS	61
Seguridad del software de virtualización	61
Seguridad del servidor virtual	62

Infraestructura de seguridad a nivel de aplicación	63
Denegación de sostenibilidad	63
Seguridad de usuarios finales	63
Seguridad de aplicaciones SaaS	64
Seguridad de aplicaciones PaaS	64
Conclusión de seguridad	64
Clase 6 - Monitoreo de procesos de negocio	65
Business Process Management	65
Etapas de trabajo con la metodología BPM	65
Elementos de un proceso	66
BPMS	66
BAM (Business Activity Monitoring)	66
Componentes de BAM	67
Funcionalidad de BAM	67
Métricas de un proceso de negocio	67
KPIs	67
Specific	68
Measurable	68
Attainable	69
Relevant	69
Time-bound	69
Conclusión KPIs	69
Clase 7 - SQL y NoSQL	70
Gestión de datos	70
Diferencias SQL y NoSQL	71
SQL	71
¿Qué hace un servidor SQL?	72
Arquitectura de los servidores SQL	72
Store procedures	72
Triggers	73
Reglas	73
SQL estático y dinámico	73
SQL incorporado y programado	73
Acceso concurrente	74
Transacciones	74
Estándar	75
No estándar	76
Procesamiento multiusuario	77
Niveles de cerramiento	77
Esquemas de cerramiento	78
SQL como middleware	78
Open Client	79
Teorema CAP	80
ACID vs BASE	80

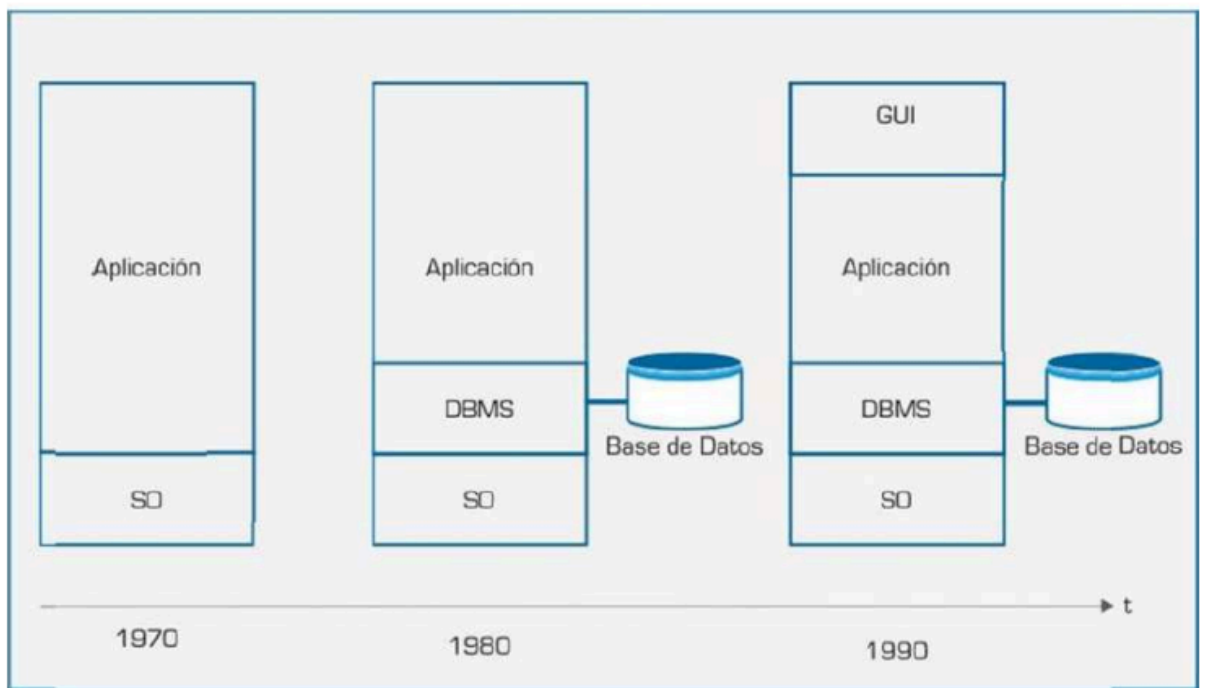
Taxonomía BBDDs NoSQL	81
Familia de columnas	81
BBDD documentales	81
MongoDB	82
BBDD orientadas a grafos	82
Clase 8 - Orquestación y coreografía BPEL	83
Impacto de SOA en la infraestructura IT	83
ESB	83
Servidor de procesos	84
Orquestación y coreografía de los procesos de negocio	84
Orquestación vs coreografía	84
BPEL	85
Orquestación de servicios web con BPEL	85
Secciones de un documento BPEL	86
Tipos de procesos BPEL	86
Composición de procesos BPEL	87
Diferencias entre BPEL y los lenguajes de programación	88
Exposición e invocación de procesos BPEL	88
Procesos BPEL sincrónicos y asincrónicos	88
Proceso de reserva de viaje en BPMN	90
Proceso de reserva de viaje en BPEL	91
Clase 9 - Ecosistemas digitales de integrabilidad	92
Transformación digital hacia el ciudadano	92
El camino hacia el gobierno inteligente	92
Principios que guían hacia el gobierno inteligente	92
Normas	93
Modelos de interoperabilidad	95
Modelado de procesos de negocio	96
Real time economy	96
Capas del modelo organizacional	97
Ruta de transformación hacia un ecosistema digital inteligente	98
¿Cómo se hace?	99
Clase de repaso	101
Evolución de los sistemas distribuidos	101
La idea de middleware	101
Servicios: un middleware posible	102
Web services	102
REST y SOAP	102
SOAP en Java	103
Los procesos de negocio como orquestadores de servicios	103
Etapas de trabajo con la metodología BPM	103
BAM vs BI	104
Ejecución de procesos: BPEL	104
ESB y motor de procesos	104

Bonita	105
Servidores SQL y NoSQL	106
SQL vs NoSQL	106
Servidores SQL	106
Modelo de computación en la nube	107
Seguridad en la nube	107
Ecosistemas Digitales de Integrabilidad	107

Clase 1 - Sistemas distribuidos

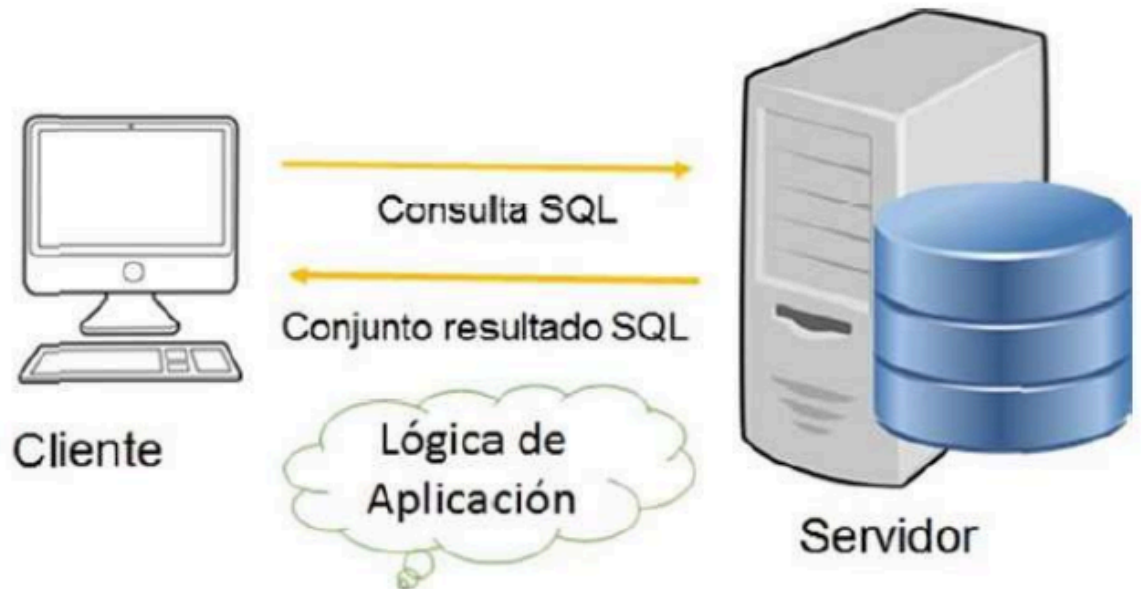
Evolución de los Sistemas de Información

Arquitectura Monolítica



Arquitectura Distribuida (2 capas y N capas)

2 capas



Surge en los 90 como solución para potenciar el poder de las pc's y concentrar funcionalidad a nivel de "servidor"

Cliente/servidor

La arquitectura Cliente/Servidor constituye uno de los primeros paradigmas "distribuidos"

Ventajas:

- Aumento del poder de las PC reduciendo costos
- Permite que el procesamiento resida cerca de la fuente de datos reduciendo el tráfico de red
- Facilita el uso de GUI
- Acepta el desafío de los sistemas abiertos
- Favorece el diseño modular

Desventajas:

- Un desbalance de tareas entre cliente y servidor puede provocar cuellos de botella

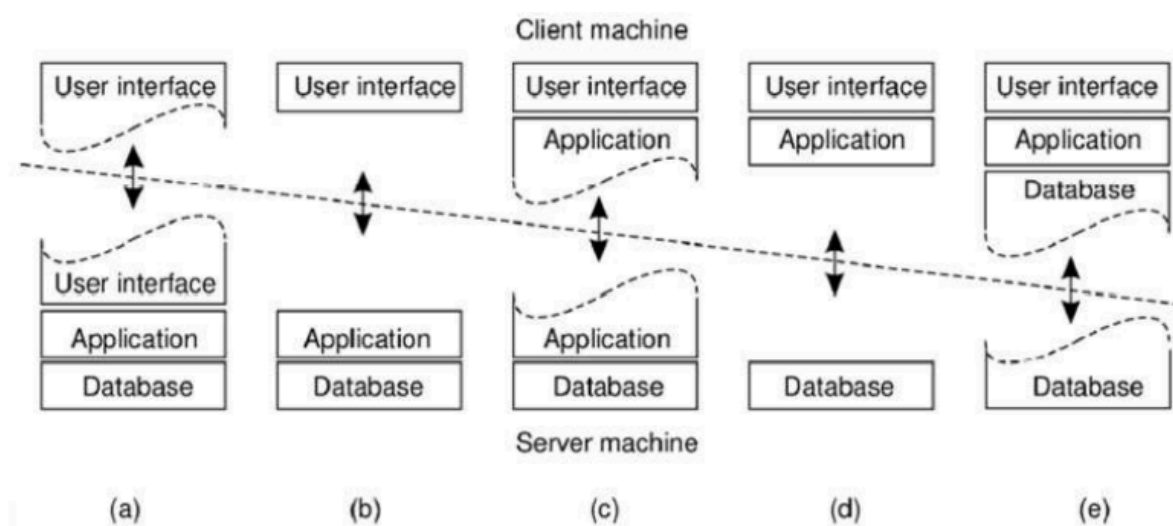
- El desarrollo de aplicaciones es más complejo creciendo también la complejidad de las herramientas de desarrollo y programación

¿Qué es?

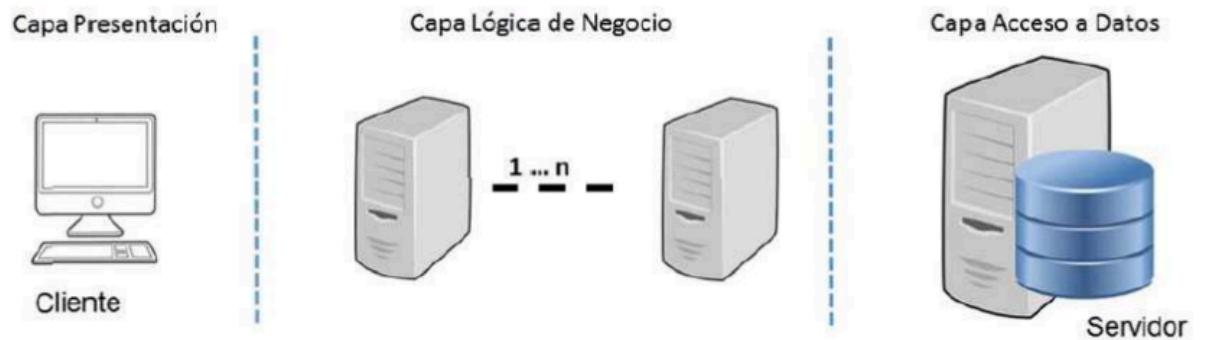
- Servicio
- Recursos compartidos
- Protocolos asimétricos
- Transparencia de la ubicación
- Intercambios basados en mensajes
- Encapsulamiento de servicios
- Escalabilidad
- Integridad

Variantes:

- File Servers
- Database Servers
- Transaction Servers
- GroupWare Servers
- Object Servers



N capas

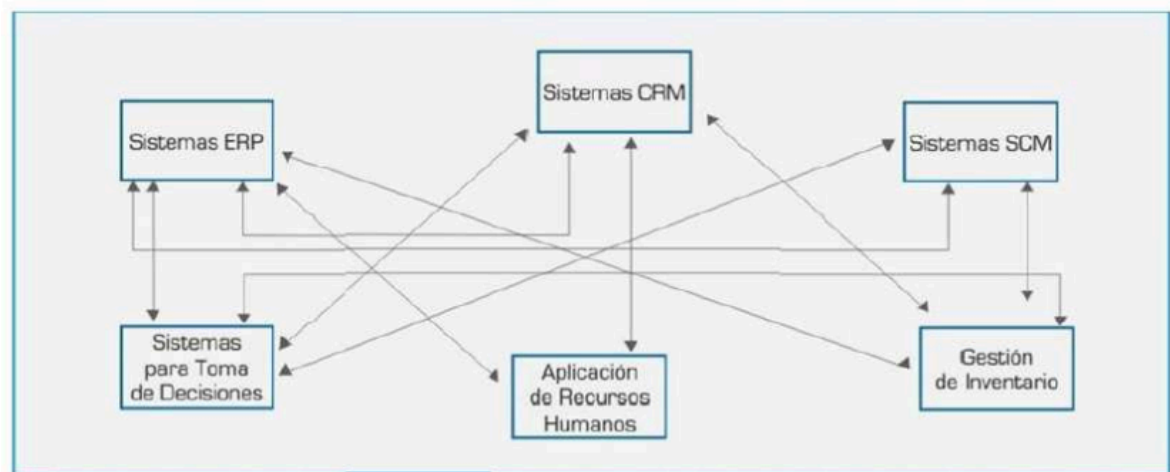


Arquitecturas Distribuidas con objetos (estándar CORBA)

No se menciona nada más en la teoría

Integración de aplicaciones

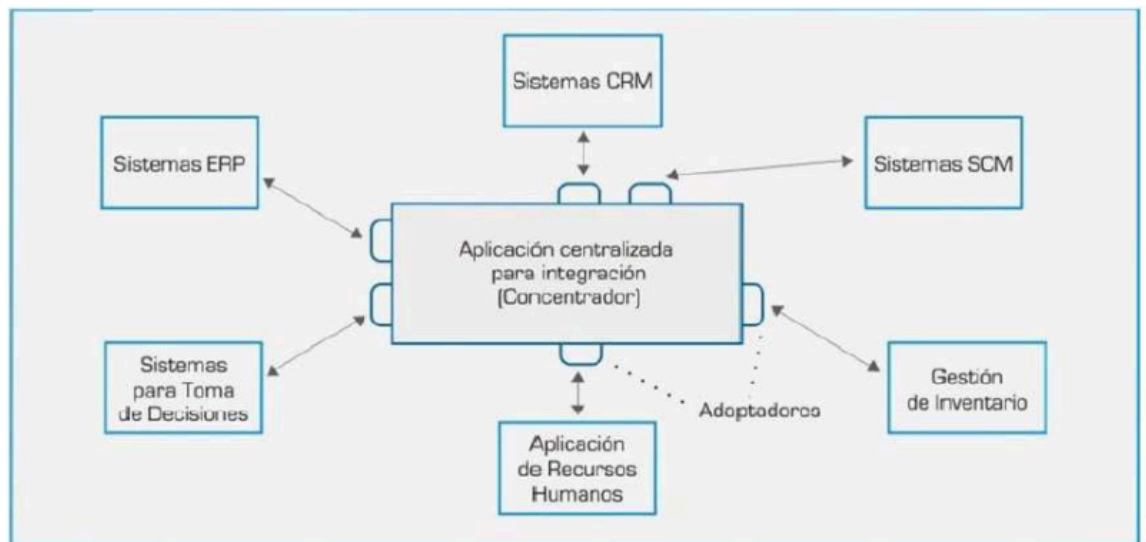
Integración punto a punto



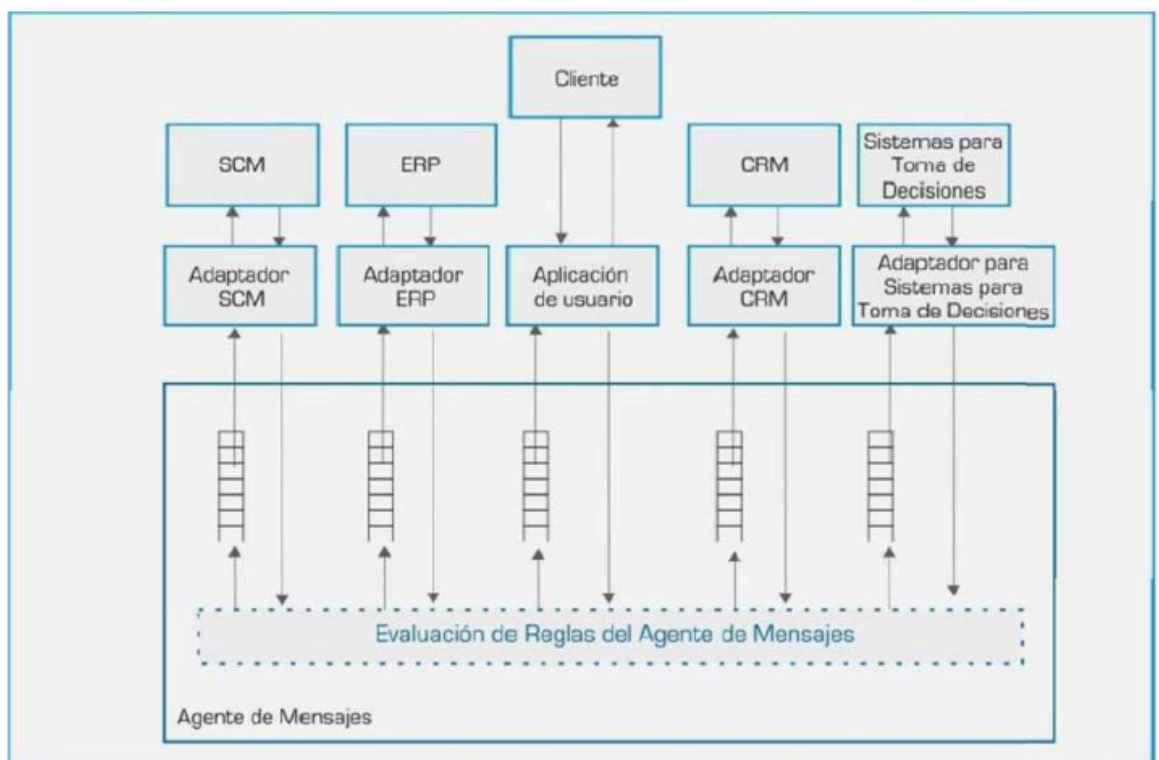
Inconvenientes:

- Arquitectura accidental (síncrona, de grano fino y no escalable)
- Dificulta los cambios: nuevos canales, nuevos productos
- Mantenimiento lento y caro
- Costoso de gestionar, monitorear y extender

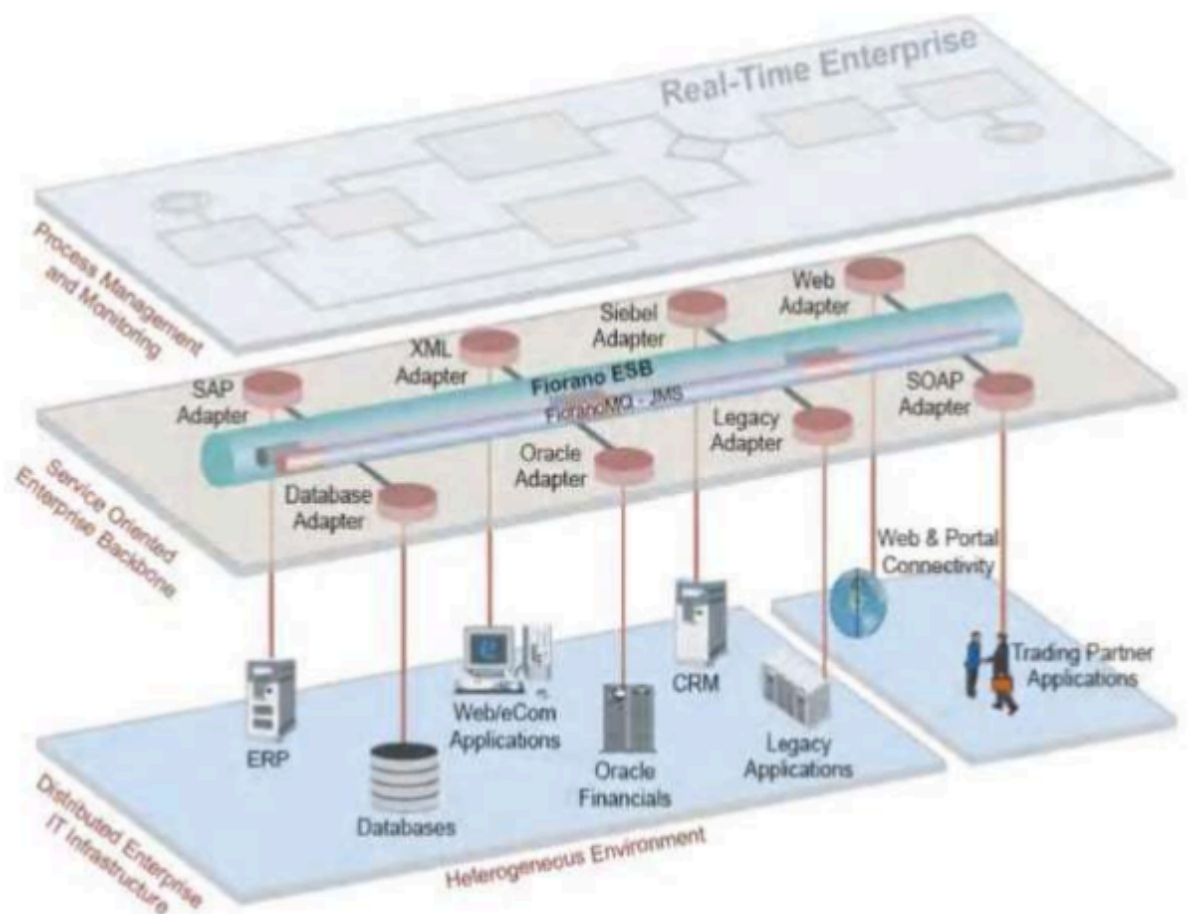
Por adaptadores



Por bus o mediador



Arquitecturas Orientadas a Servicios



Definiciones de sistemas distribuidos

- Tanenbaum: A distributed system is a collection of independent computers that appears to its users as a single coherent system
- Colouris: A distributed system is one in which components located at networked computers communicate and coordinate their actions only by passing messages

Middleware

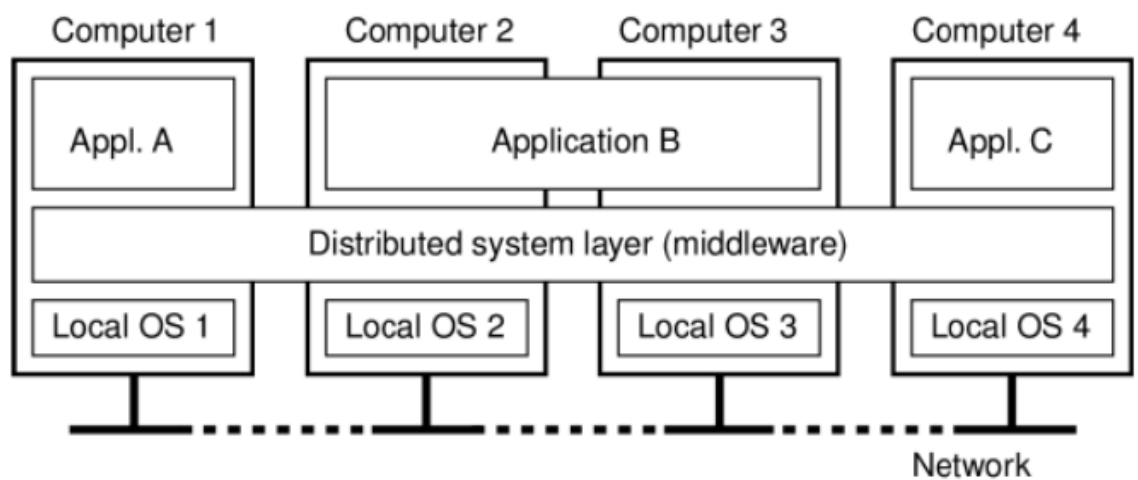
El MiddleWare comienza con la porción de cliente que es usada para invocar el servicio, cubre la transmisión sobre la red y retorna el resultado

Existen dos grandes clases de middleware:

- General:
 - Transporte

- Red
- Específico del servicio:
 - RPC
 - SQL
 - Web Services
 - MOM

Objetivo: que las computadoras se mantengan independientes pero que el sistema siga siendo único



Middleware para la comunicación/componente middleware (porque para ambos títulos se usa la misma imagen):



Componente middleware

¿Cuál es la diferencia con middleware? Ni idea

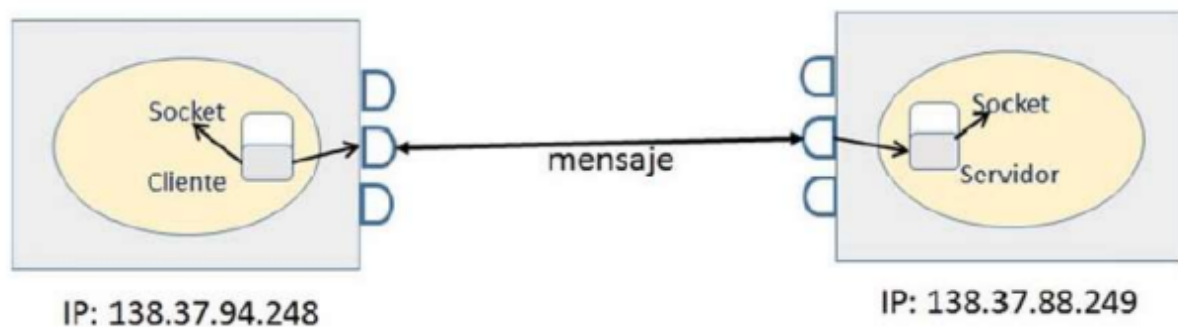
Corre en ambos lados de la aplicación, el lado del cliente y el del servidor

Dividimos este componente en tres categorías:

- Sistema operativo de red
- Protocolo de transporte
- Específico del servicio

Modelos de middleware

Middleware por socket



Las operaciones de los sockets son:

1. Se crea un socket con la función socket
2. Los clientes se conectan con connect
3. Los servidores aceptan conexiones de clientes con la función accept
4. Clientes y servidores intercambian datos con read y write
5. Las conexiones se cierran con close

Se pierde mucho tiempo haciendo o preparando la conexión, solo el punto 4 es de comunicación de datos

Middleware por RPC o RMI

El ambiente de programación RPC

El nacimiento del RPC corresponde al año 84 y especifica una manera diferente de trabajar sobre las redes que provoque el mismo efecto que se obtiene con la programación en un ambiente único con la idea de programa y subrutina

La idea nueva es obtener aplicaciones que funcionen exactamente igual en un ambiente centralizado que en un ambiente distribuido

Características:

- Pasaje de parámetros
- Binding
- Tratamiento de excepciones
- Transparencia de la representación de datos

Es el middleware específico del servicio más alejado del OS

Middleware orientado a mensajes

MOM (Message-Oriented Middleware) permite intercambio de mensajes de cualquier tipo entre cliente y servidor donde dichos mensajes son encolados

Las aplicaciones se comunican poniendo mensajes en una cola de mensajes

Características:

- Oculta totalmente los detalles físicos de la red y la comunicación se realiza sin saber si existe un enlace

- Los programas no hablan unos con otros en forma sincrónica

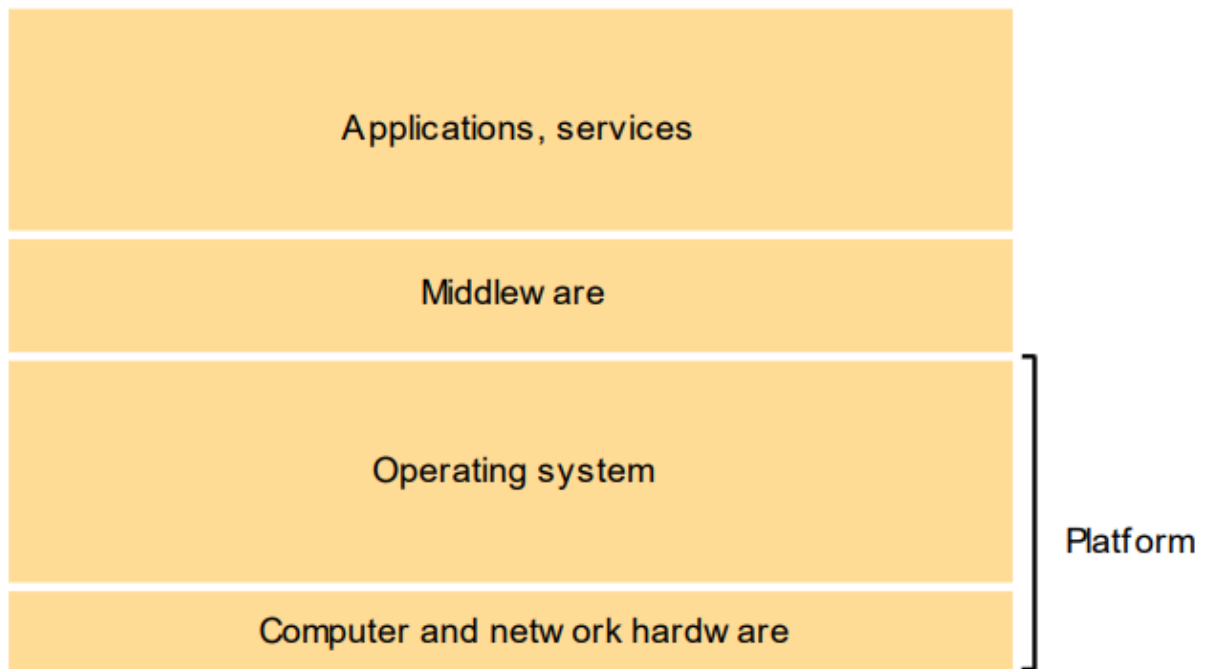
MOM vs RPC

Característica	MOM	RPC
Metafora	Como oficina postal	Como telefono
Tiempo de relacion C/S	Asincronico. Clientes y servidores operan en distinto tiempo	Sincronico. Clientes y servidores corren a la vez y se esperan
Secuenciamiento C/S	Sin secuencia fija	Los servidores se lanzan antes que los clientes
Estilo	Cola	Call/Return
Datos persistentes	SI	NO
Compañero disponible	NO	SI
Balanceo de carga	Una cola simple FIFO o con prioridades	Requiere un monitor de TP adicional
Soporte transaccional	SI (algunos productos)	NO (requiere RPC transaccional)
Flitrado de mensajes	SI	NO
Performance	Baja	Alta
Procesamiento asincrónico	SI (se requiere colas y triggers)	Limitado (requiere hilos y trucos de programación)

Arquitectura distribuida

A grandes rasgos un arquitectura distribuida consiste de cuatro niveles:

- Nivel de aplicación
- Nivel de sistema operativo distribuido
- Nivel de manejo local
- Nivel del sistema de comunicación



El protocolo de transporte del middleware

APLICACIÓN	MOM	RPC		Peer-to-Peer
		Named Pipes	DCE	
PRESENTACION		NDR	SXDR	
SESION	NETBIOS / SOCKETS / TLI			
TRANSPORTE	RED	TCP/IP	IPX/SPX	
RED				
LLC		802.2		
MAC	TOKEN RING/ETHERNET		SDLC	
FISICA	FIBRA OPTICA	COAXIL	TELEFONO	

Servicios de seguridad distribuidos

El ambiente cliente/servidor introduce un nuevo tratamiento de la seguridad respecto del que se tenía en los sistemas tradicionales

En un ambiente cliente/servidor no se puede asegurar que cualquier sistema operativo sobre la red proteja los recursos del servidor de acceso no autorizados y aunque se contara con clientes totalmente seguros, la red en sí es un medio altamente violable e inseguro

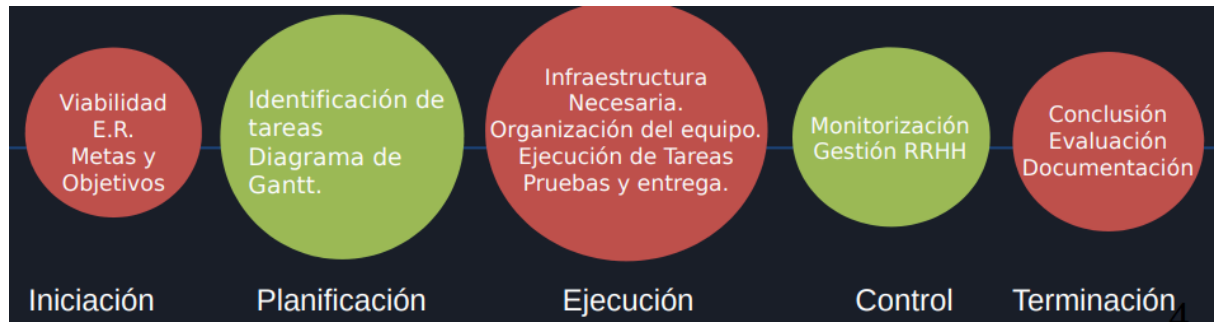
Nivel C2 de seguridad

- Autenticación: (soy quien digo ser)
- Autorización: (sólo uso lo que me es permitido)
- Auditoría: (qué hice)

Clase 2 - Gestión de proyecto

Involucra planificación, monitoreo, control de los recursos humanos, control del proceso y actividades que se dan conforme el software evoluciona

Esto involucra a todos los involucrados (grandes palabraaaaaas 😊), cada uno con sus tareas específicas



Clase 3 - Procesos de negocio

Un proceso de negocio es un conjunto de actividades que se realizan en coordinación en un ambiente organizacional y técnico

Estas actividades alcanzan el objetivo del negocio

Cada proceso de negocio representa una única organización, pudiendo interactuar con otras

Orientación de una organización a procesos de negocio

La orientación a Procesos permite:

- Organizar el funcionamiento de la empresa
- Evitar organizarlo por las necesidades de las áreas funcionales

Orientada a Funciones	Orientada a procesos
Los roles y las responsabilidades están alineadas por áreas.	Los roles y las responsabilidades están alineadas por Procesos de Negocio.
No hay una visibilidad clara del proceso a través de las áreas funcionales.	La visibilidad del proceso de negocio se tiene de principio a fin.
El control de costes se realiza por centros de costes alineados a áreas funcionales.	Se pierde el valor del proceso al dar más peso a la búsqueda de eficiencia de las áreas funcionales.
El control de costos se alinea por pasos de los procesos, se facilita una gestión ABM (Activity Based Management).	Se centra en la eficacia, sin perder eficiencia, facilitando la generación de valor a los clientes.

Beneficios obtenidos para la organización

- Efectividad: se concreta lo que ha de llevarse a cabo
- Eficiencia
- Consistencia
- Prerrequisitos
- Productividad
- Ahorro
- Calidad

Beneficios obtenidos para el analista

- Agilización del proceso de desarrollo
- Identificación de errores
- Mayor nivel de abstracción
- Independencia de plataformas tecnológicas concretas
- Trazabilidad del sistema

BPM

La gestión de procesos de negocio se basa en la idea de que cada producto es el resultado de un conjunto de actividades realizadas para obtenerlo

BPM incluye conceptos, métodos y técnicas para soportar el diseño, administración, configuración, representación y análisis de los procesos de negocio

Su objetivo último es representar el proceso de negocio con sus actividades y las restricciones de ejecución entre ellas

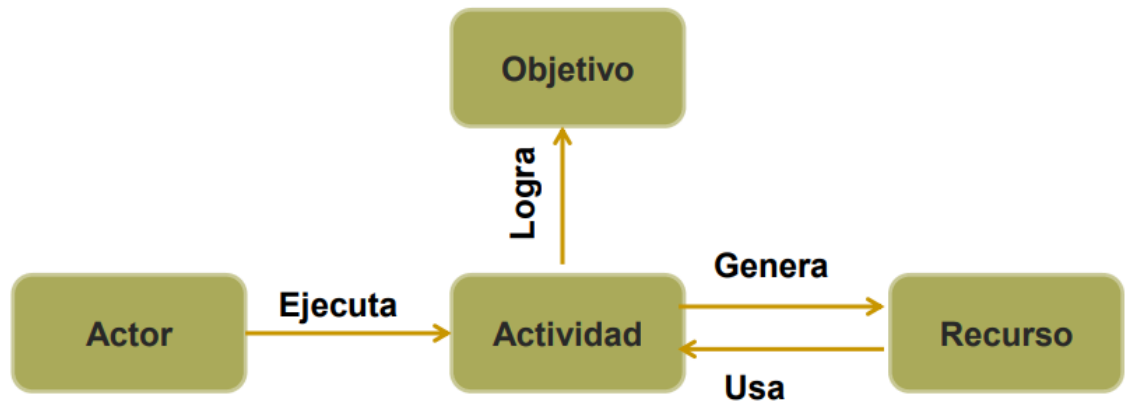
Proceso

Un proceso es:

- Un ordenamiento específico de actividades a través de tiempo y lugar
- Con un principio y un final
- Con entradas y salidas claramente identificadas

Elementos

- Mapa y diagrama de procesos
- Actores y propietario
- Reglas de negocio
- Indicadores (eficacia, eficiencia, rastros de tiempo)
- Subprocesos o actividades clave



Mapa y diagrama de procesos

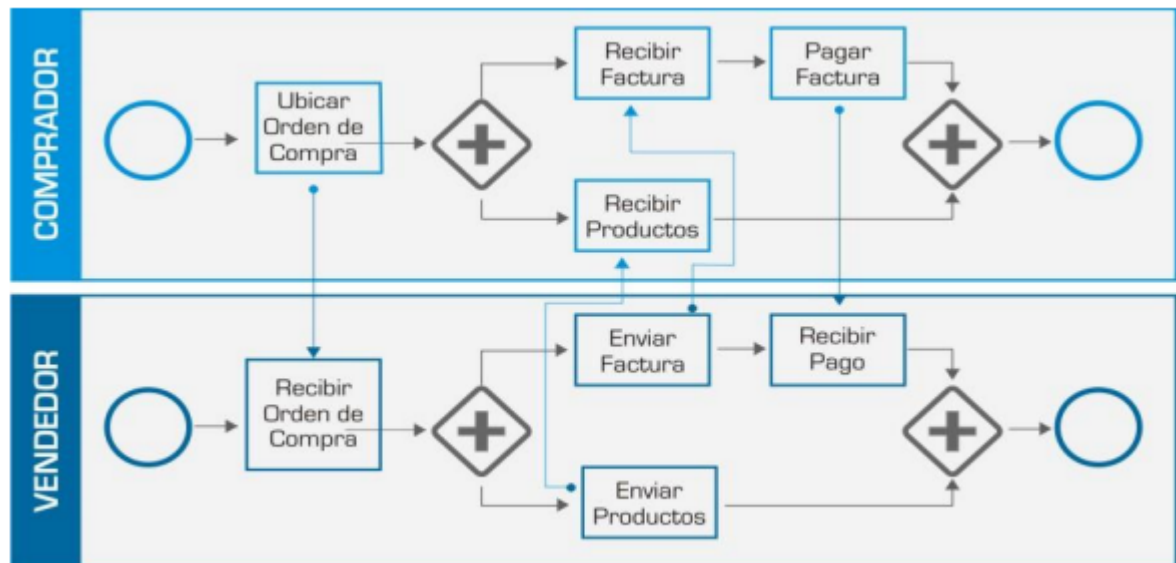
Un mapa de proceso es una representación gráfica de la organización en términos de su funcionamiento

Un diagrama de procesos describe en forma detallada actividades, eventos, secuencias y resultados del mismo

Una instancia de proceso de negocio representa un caso concreto dentro de los procesos operativos de una compañía. Estos procesos se componen de un conjunto de instancias de actividades

Elementos básicos de un BPD (BP Diagram)

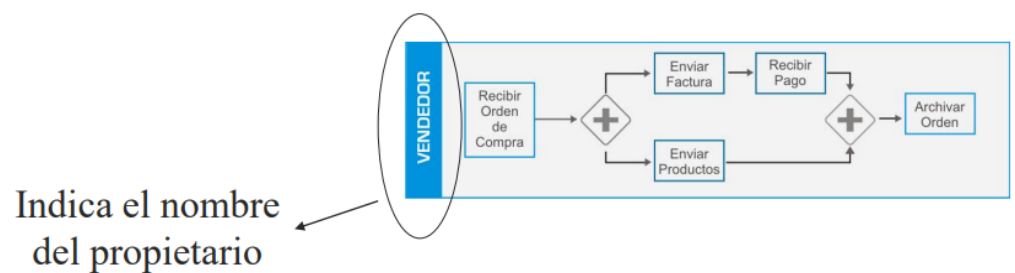
- Flujos de entrada y salida (eventos y resultados)
- Actividades
- Secuencias
- Puntos de decisión
- Participantes



Actores y propietarios

Las actividades de los procesos son responsabilidad de personas o áreas de la organización, deben tener un propietario

Las responsabilidades pueden darse en varias fases del ciclo de vida del proceso

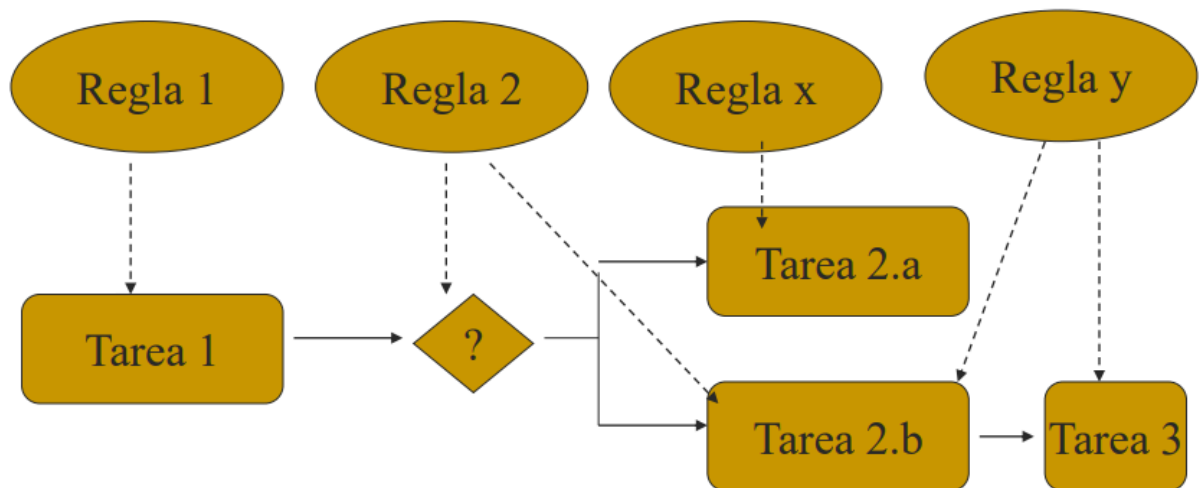


Reglas de negocio

Las reglas de negocio se establecen para determinar cómo opera la organización para cumplir sus objetivos

Se aplican en varios pasos de un proceso

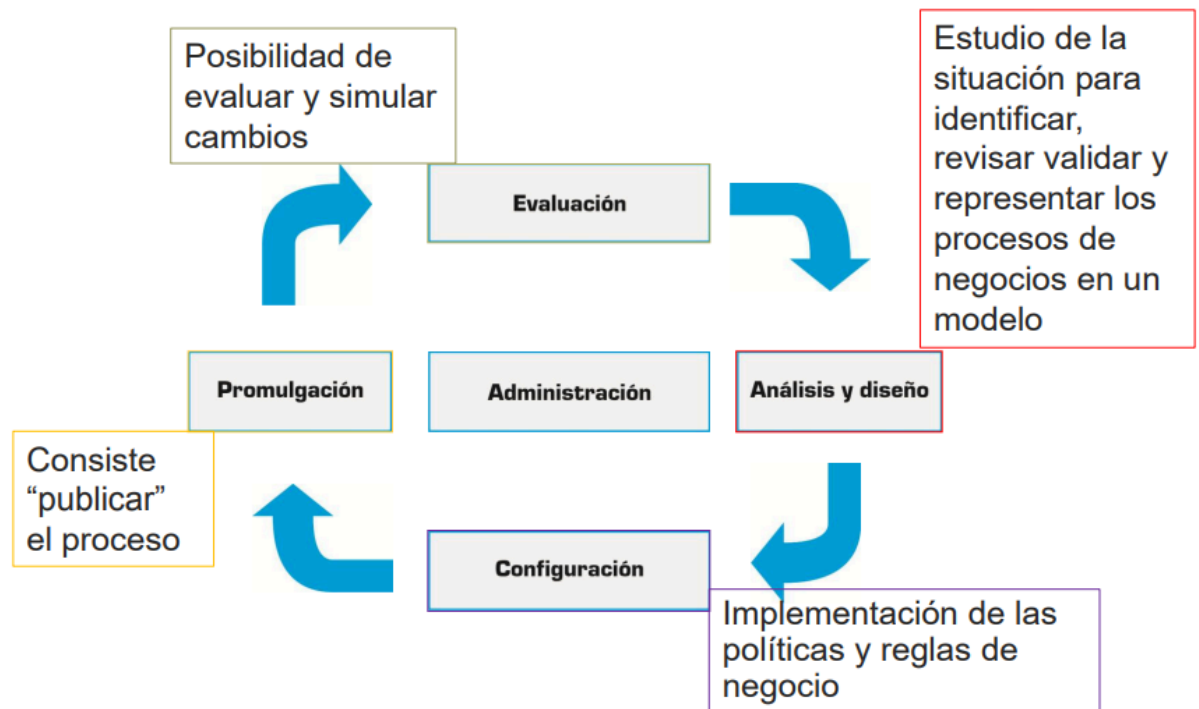
Es preciso definirlas en forma separada del proceso en sí



Ciclo de vida clásico



Ciclo de vida según M. Weske



BPMS

Los sistemas de gestión de procesos de negocios (BPMS) son softwares utilizados como herramientas para representar y coordinar las actividades involucradas en un proceso de negocio

Convierten el modelo del proceso en un proceso ejecutable

Componentes tecnológicos de los BPMS

- Motores de orquestación: guían la secuencia de actividades
- Herramientas de análisis y Business Intelligence: estudian la información resultante de la ejecución del proceso en tiempo real
- Motores de reglas: ejecutan reglas que permiten abstraer las políticas y decisiones de negocio
- Repositorios: almacenan los componentes y recursos de los procesos disponibles para su reutilización en múltiples procesos
- Herramientas de simulación y optimización: permiten a los administradores del negocio comparar el nuevo diseño de procesos

- Herramientas de Integración: permiten integrar el modelo con otros sistemas

Aporte de los BPMS

Automatizan las actividades del ciclo de vida y facilitan la gestión de los procesos de negocio permitiendo:

- Adherencia a estándares (BPMN)
- Ejecución directa de los procesos modelados en un lenguaje de ejecución (BPEL)
- Monitoreo en tiempo real (BAM)

BAM

Bajo un BPMS, la monitorización de los procesos permite obtener en tiempo real información de los indicadores clave que se hayan definido con objeto de contar con un soporte para su mejora continua

Clase 4 - Servicios web

Evolución de las aplicaciones

- Aplicación Monolítica
- Arquitectura Distribuida (2 capas y N capas)
- Arquitecturas Distribuidas con objetos (estándar CORBA)
- Integración de aplicaciones
- Arquitecturas Orientadas a Servicios

Arquitectura basada en servicios SOA

Las SOA están formadas por servicios de aplicación débilmente acoplados y altamente interoperables

Se basa en tres conceptos técnicos:

- Servicios:
 - Son piezas funcionales que resuelven un aspecto del negocio
 - Pueden ser simples (almacenar los datos de un cliente) o compuestos (el proceso de compra de un cliente)
- Interoperabilidad a través de un ESB (Enterprise Service Bus):
 - Es la infraestructura que habilita una alta interoperabilidad entre servicios en un contexto distribuido
- Bajo acoplamiento:
 - Concepto de reducir las dependencias entre sistemas
 - Es una idea muy difícil de desplegar, mantener y corregir

Es un paradigma capaz de organizar y utilizar capacidades distribuidas bajo una o más organizaciones, donde es posible proveer un medio uniforme para publicar, descubrir, interactuar y utilizar mecanismos que logren un objetivo

Características de SOA

- Reuso de componentes de software existentes
- Interoperabilidad entre aplicaciones y tecnologías heterogéneas
- Flexibilidad para componer, integrar y escalar soluciones

Principios de diseño SOA

- Contrato de servicio estándar:
 - Expresan el propósito del servicio
 - Por ejemplo: WSDL, Esquema XML, IDL
- Bajo acoplamiento del servicio:
 - El acoplamiento mide el nivel de conexión entre dos elementos
 - El bajo acoplamiento de un servicio está dado por la independencia entre las prestaciones del servicio y su implementación
- Abstracción del servicio:
 - Promueve el ocultamiento de detalles y las interfaces mínimas
- Reusabilidad del servicio:
 - Servicio como recurso independiente del contexto funcional
- Autonomía del servicio
 - Los servicios deben tener el control sobre su entorno y recursos
- Falta de estado del servicio (stateless):
 - La gestión del estado de un servicio puede comprometer su disponibilidad
- Capacidad de encontrar el servicio
 - Los servicios deben definir: propósito, capacidad y limitación de recursos, independientemente de la modalidad de registro
- Componibilidad del servicio:
 - La composición de servicios permite escalar la complejidad de las arquitecturas SOA

Actores SOA



Servicios web

Nuevo paradigma de desarrollo de aplicaciones interoperables entre sí y basadas en internet

Permiten que las compañías vinculen el software

Middleware que provee una interfaz entre clientes y servidores

Los clientes acceden a las operaciones publicadas en la interfaz del servicio web

Características de los servicios

- Son el bloque fundamental de una SOA
- Poseen interfaces bien definidas, independientes de su implementación
- Los clientes consumen los servicios sin la necesidad de entender cómo estos ejecutan sus pedidos
- Los servicios pueden ser descubiertos dinámicamente
- Los servicios pueden componerse
- Proveen funcionalidad de grano grueso (ofrecen mayor funcionalidad, más abarcativas, ej.: un servicio de consulta del precio del dólar me da precios de distintos dólares (oficial, blue, MEP, etc.))
- Cumplen con un objetivo de negocio predeterminado
- No dependen del contexto o estado de otros servicios

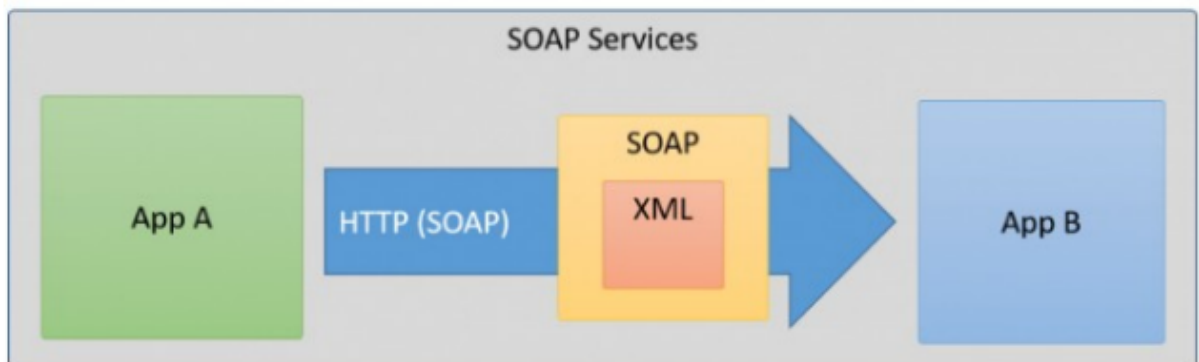
- Transparencia de localización (locationally transparent)

SOAP vs REST

Tanto SOAP como REST son tecnologías que implementan la arquitectura SOA

	Rest	SOAP
Formato del mensaje	XML, JSON, etc.	XML dentro de SOAP
Definición de la interfaz	No es necesario	WSDL
Transporte	HTTP	HTTP, JMS, FTP, etc.

SOAP



Comunicación bajo el protocolo SOAP

Solo soporta formato XML ⇒ menor flexibilidad

Los servicios web SOAP utilizan SOAP para:

- Invocar WS a través de interfaces descritas en WSDL
- Codificar los parámetros de entrada y de salida en XML

Para ello deben:

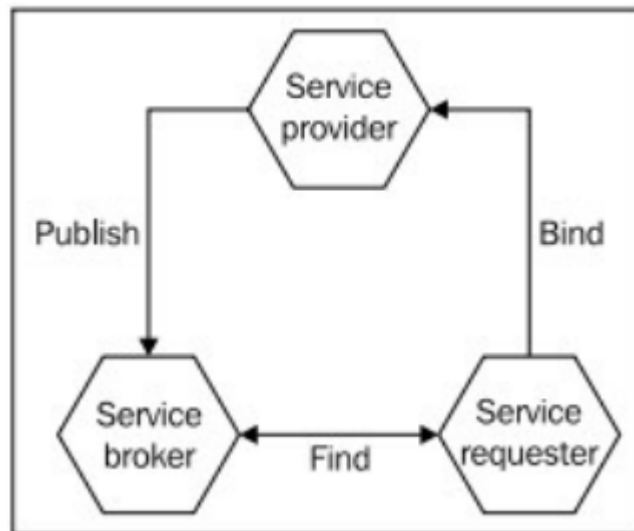
- Traducir las peticiones de servicios en invocaciones de lógica
- Traducir los parámetros entre distintos lenguajes de programación

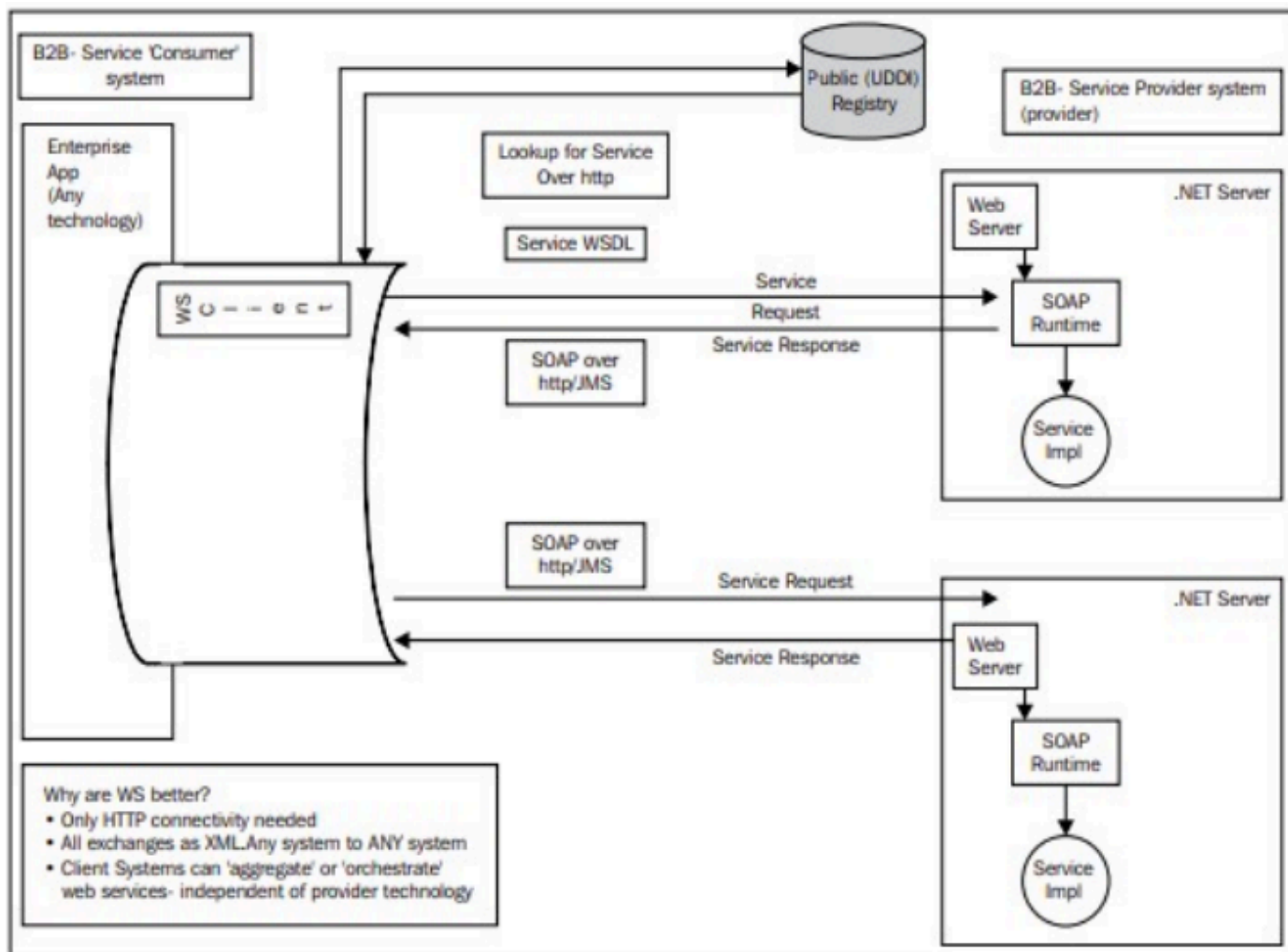
Requisitos

1. Definir el servicio web
2. Publicar el servicio web
3. Debe poder ubicarse
4. Invocar el servicio web
5. Despublicar en caso de ser necesario

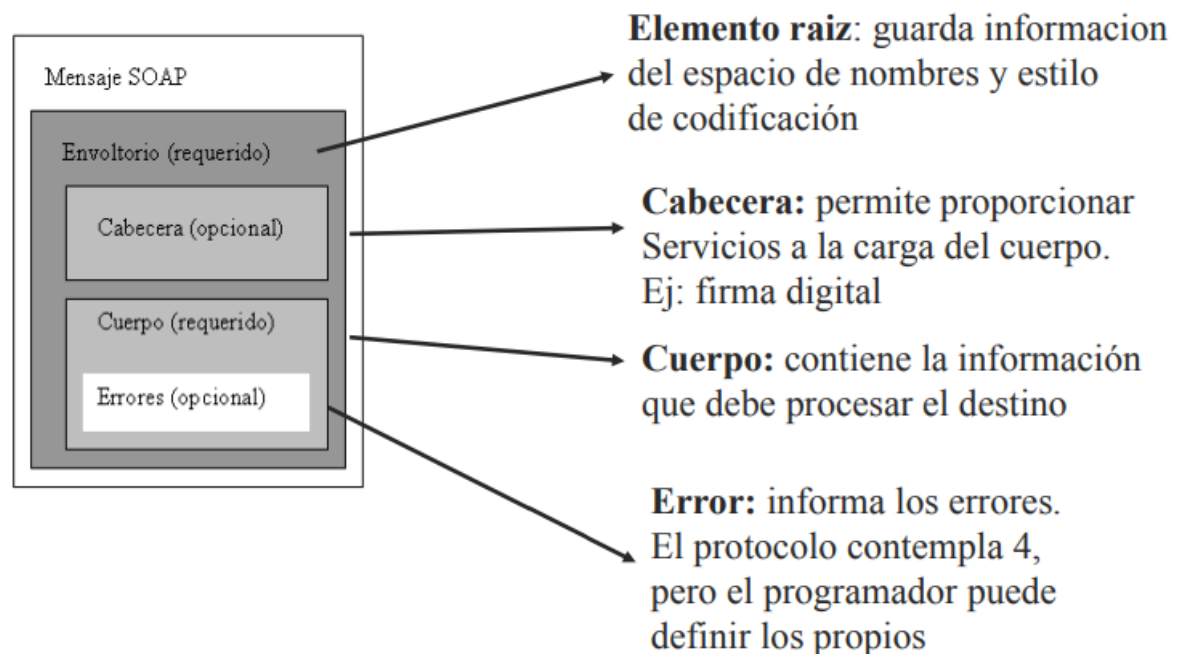
Descripción del Servicio: WSDL

Descubrir el Servicio: UDDI





Estructura de un mensaje SOAP



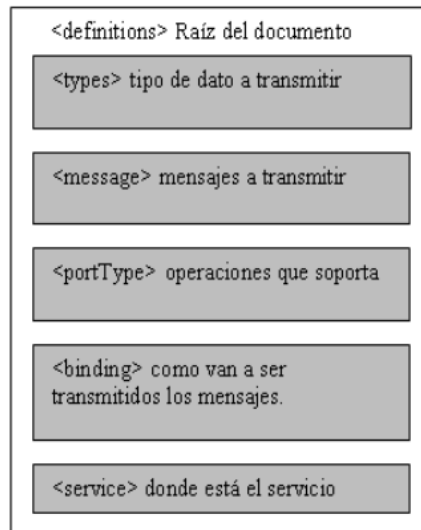
<i>faultCode</i>	Código de texto utilizado para indicar la clase de error.
<i>faultString</i>	Texto para explicación del error.
<i>faultActor</i>	Indica quien causo el error. Esto es útil en caso de que el mensaje SOAP viaje a través de múltiples nodos.
<i>detail</i>	Este elemento permite contener mensajes de errores específicos de la aplicación.

Estructura de codificación de errores

Pila de protocolos

Ofrece un directorio de servicios en Internet	UDDI Encontrar
Ofrece un modo de definir los servicios	WSDL Describir
Permite invocar métodos de los servicios.	SOAP Invocar
Permite a los consumidores de servicios enviar y recibir mensajes a y de los servicios	XML y XML Schema Datos
Son protocolos abiertos de Internet. Dan soporte a las capas superiores.	HTTP, SMTP, TCP... Transporte

Estructura de un documento WSDL



Definitions: es la raíz de cualquier documento WSDL. Define nombre del Servicio Web y los múltiples namespaces a ser utilizados por el documento.

Types: define los tipos de datos usados por el cliente y el servidor.

Message: define el nombre del mensaje y las partes del mismo que pueden ser parámetros o valores de retorno.

PortType: define las diferentes invocaciones a servicios y el patrón de comunicación que se utilizara para el intercambio de mensajes.

Binding: describe de que forma el servicio va a ser comunicado.

Service: determina la dirección en la cual se invocara el servicio.

El protocolo UDDI

UDDI es en sí mismo un servicio web que establece un registro de búsqueda de servicios organizado de manera jerárquica

Consta de dos partes:

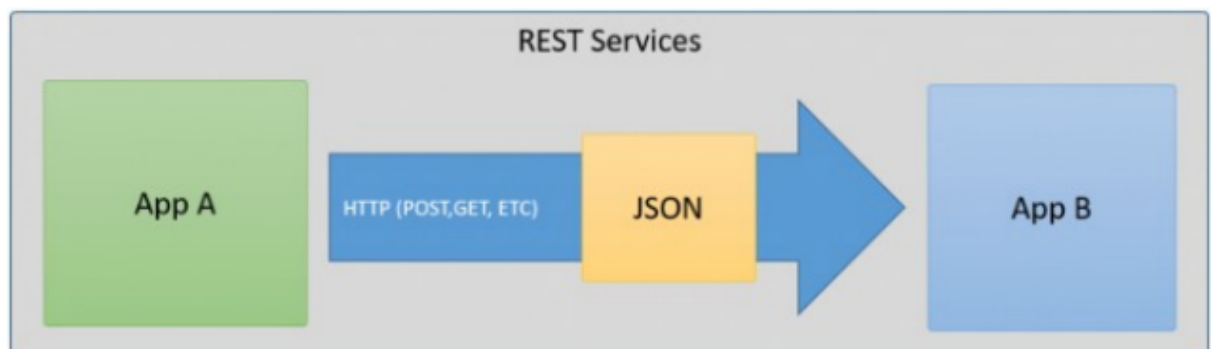
1. Un registro de información de los Servicios Web y el documento WSDL asociado a cada uno de ellos
2. Un conjunto de documentos WSDL utilizados para manipular y buscar sobre el registro mismo

La información que se almacena en el registro UDDI se clasifica en tres formas:

- Páginas blancas: contiene información general de quien publica el servicio
- Páginas amarillas: contiene información clasificada por tipo de negocio, como productos y servicios que comercializa
- Páginas verdes: contiene información técnica acerca de los servicios web que expone

La descripción de servicios web en UDDI no está limitada únicamente a servicios basados en SOAP, sino también servicios CORBA, RMI, e incluso simples páginas web

RESTful



Las arquitecturas REST se construyen sobre recursos identificados por URIs

Tienen una interfaz común, por lo que las únicas operaciones permitidas son GET, POST, PUT y DELETE

No tienen estado ni acceden a información de contexto

Toda la información para entender la petición debe encontrarse en la misma

Utiliza códigos de respuesta nativos (2xx, 3xx, etc.)

Permite transmitir cualquier tipo de datos, XML, JSON, Text

Reglas de diseño

Deben ser stateless, dado que cada solicitud realizada por un cliente debe ser siempre independiente de la anterior

Utilizar el verbo HTTP correcto. No solo GET y POST, también PUT, PATCH, DELETE u OPTIONS

Utilizar nombres de recursos apropiados

Utilizar JSON como respuesta

Considerar la conectividad

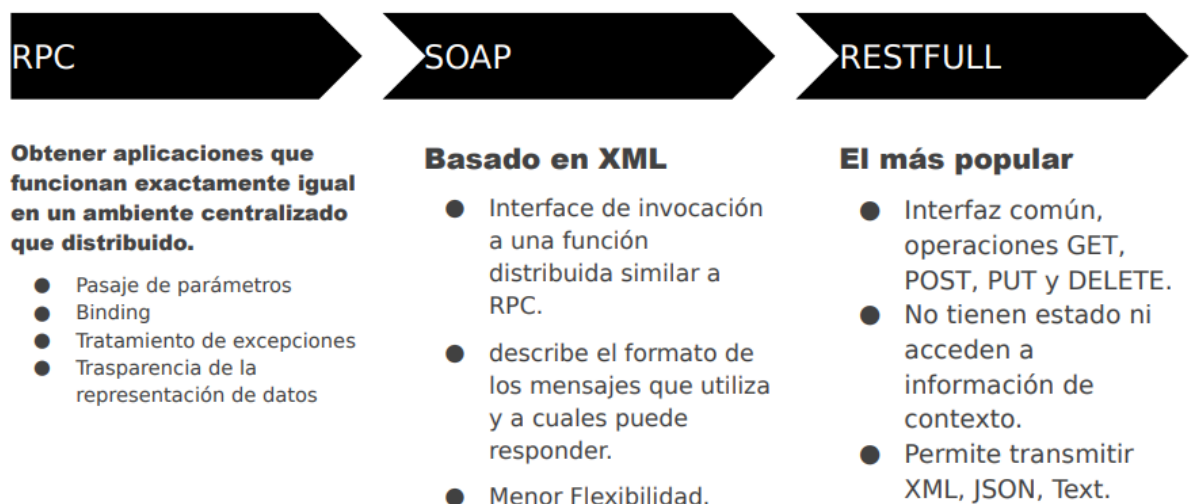
Utilizar SSL para todas la peticiones

Cada solicitud debe enviarse con un mecanismo de autenticación, como por ejemplo OAuth 2

GraphQL

GraphQL es un lenguaje de consulta de datos, pero también un entorno de ejecución, porque permite llevar a cabo las consultas responsabilizándose del análisis y validación de estas además de la serialización de las respuestas (es decir, convertir el objeto a devolver)

Conceptos previos



Problemas de REST

- Over-fetching (sobrecarga): una respuesta supera las necesidades de datos de nuestra aplicación
- Under-fetching (captura ineficiente): varias consultas para obtener una cantidad de información necesaria
- Gestión de endpoints: a medida que cambian las necesidades del cliente, normalmente tiene que crear nuevos endpoints, y esos endpoints pueden comenzar a multiplicarse rápidamente

Con GraphQL, la arquitectura típica implica un único endpoint sobre el cual definimos “qué” queremos consultar y “cómo” debe ser el formato de la respuesta

Ventajas de GraphQL

- Exactamente lo que necesito:
 - Con GraphQL se pueden restringir los datos que deben obtenerse del servidor
- Muchos recursos en una misma petición:
 - Las consultas GraphQL recuperan los objetos asociados, mientras que el REST lo hará a partir de peticiones a varias URL
- Sistema de Tipos:
 - GraphQL es fuertemente tipado
 - Las consultas se basan en campos y sus datos asociados a tipos
 - Si hay una falta de coincidencia de tipos en una consulta GraphQL, las aplicaciones del servidor devuelven un mensaje de error
- Introspección
- Control de versiones
- Ecosistema en crecimiento

Desventajas de GraphQL

- Complejidad de Consultas:
 - GraphQL no elimina los cuellos de botella de rendimiento cuando tiene que acceder a varios campos en una consulta
- Limitación de Tasa:

- Mientras que en REST se permiten una determinada cantidad de solicitudes por día, para GraphQL esto es más difícil
- Por lo general lo que se hace es reducir las profundidades de la consulta o su complejidad
- Caché:
 - En REST, se accede a los recursos con URL. Almacenar en caché es mantener la respuesta asociada a una URL
 - En GraphQL puede ser más complejo porque aunque se haga la consulta sobre las mismas entidades la consulta puede ser diferente

REST vs GraphQL

- La tecnología REST es la más utilizada y ampliamente aceptada
- Las respuestas de una API GraphQL son considerablemente más reducidas que las generadas con una API REST
- Bajo estrés, una API REST presenta mayor velocidad de respuesta
- REST impone que debe utilizarse el caché mientras que GraphQL como tecnología no ofrece de forma natural este middleware. Por esto una API REST tendrá constantemente una velocidad mayor al momento de responder peticiones comunes
- GraphQL es mucho más ventajoso en las áreas de under-fetching y over-fetching
- En general, desde el punto de vista de rendimiento, GraphQL debería utilizarse cuando se requiere menor utilización de la memoria, cuando los recursos que la API maneja están estrechamente relacionados entre sí y cuando no se considere que vaya a utilizarse la data completa de los recursos
- REST debería utilizarse en caso de necesitar mayor velocidad de respuesta, cuando los recursos no están relacionados entre sí de una forma tan estrecha y cuando se vaya a utilizar toda la data de cada uno de los recursos manejados

Principios de diseño

- Jerárquico:
 - Una consulta GraphQL en sí está estructurada jerárquicamente
 - La consulta tiene la misma forma que los datos que devuelve
- Centrado en el producto:
 - Se rige por los requisitos de las vistas y los ingenieros de front-end que las escriben

- Tipado fuerte:
 - Cada servidor GraphQL define un sistema de tipos específico de la aplicación
 - Las consultas se ejecutan dentro del contexto de ese sistema de tipos
- Consultas especificadas por el cliente:
 - GraphQL publica las capacidades que sus clientes pueden consumir
 - Es el cliente especifica exactamente cómo consumirá esas capacidades publicadas
- Introspectivo:
 - El propio lenguaje GraphQL debe poder consultar el sistema de tipos de un servidor GraphQL

Conclusiones

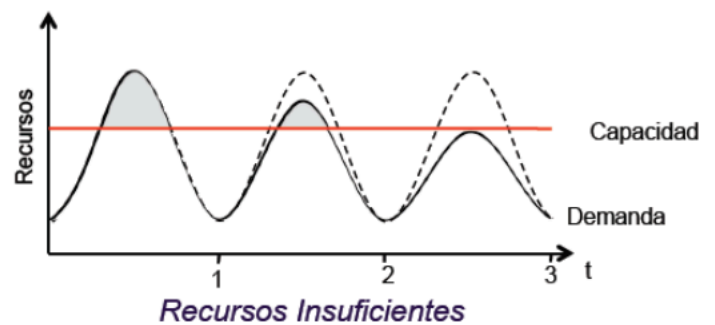
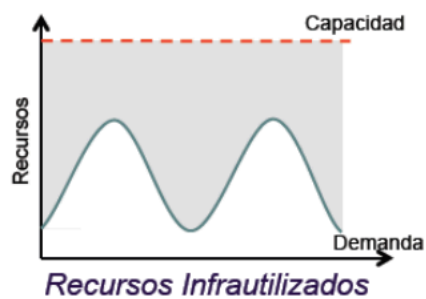
- Definición:
 - GraphQL es un lenguaje de consulta de datos, pero también un entorno de ejecución, porque permite llevar a cabo las consultas
 - Podemos consultar “qué queremos” y “cómo lo queremos”
- Ventajas:
 - Único Endpoint
 - Fuertemente Tipado
 - (Over/Under)fetching
- Desventajas:
 - No elimina cuellos de botella
 - Caching
- Comparación con REST:
 - GraphQL debería utilizarse:
 - Cuando se requiere menor utilización de la memoria
 - Cuando los recursos que la API maneja están estrechamente relacionados entre sí
 - Cuando no se considere que vaya a utilizarse la data completa de los recursos.

Clase 5 - Cloud computing

La Computación Cloud es un paradigma que posibilita el acceso ubicuo bajo demanda a servicios TIC accesibles a través de Internet

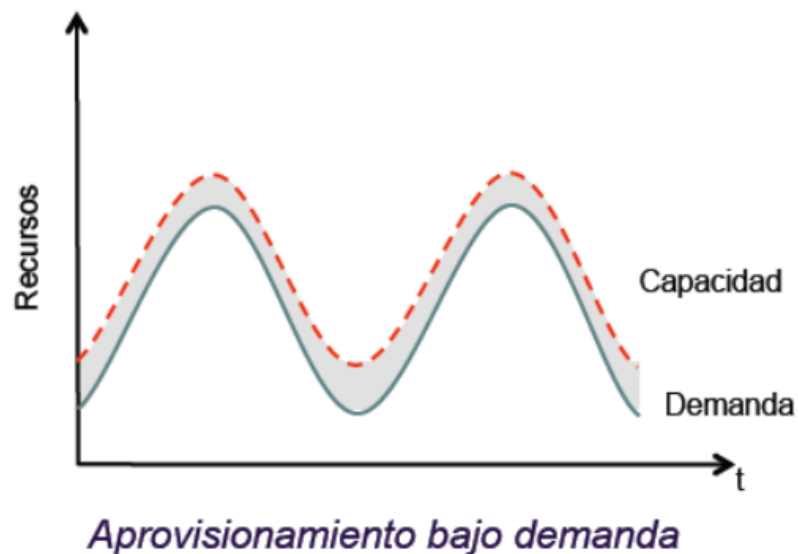
El término cloud (nube) se refiere a la forma de representar la red (Internet) en los diagramas y es una abstracción de las complejidades de su infraestructura

El desafío es cómo dimensionar la capacidad de los servidores para ajustarlos a la demanda (servicios estacionales, eventos masivos y puntuales, juegos on line)



La idea es contar con aprovisionamiento bajo demanda:

- Consumí lo que necesitás
- Pagá por lo que consumís



Datacenters

Esta evolución también ha supuesto un cambio profundo en la forma en que los ISPs gestionan y ofrecen sus recursos

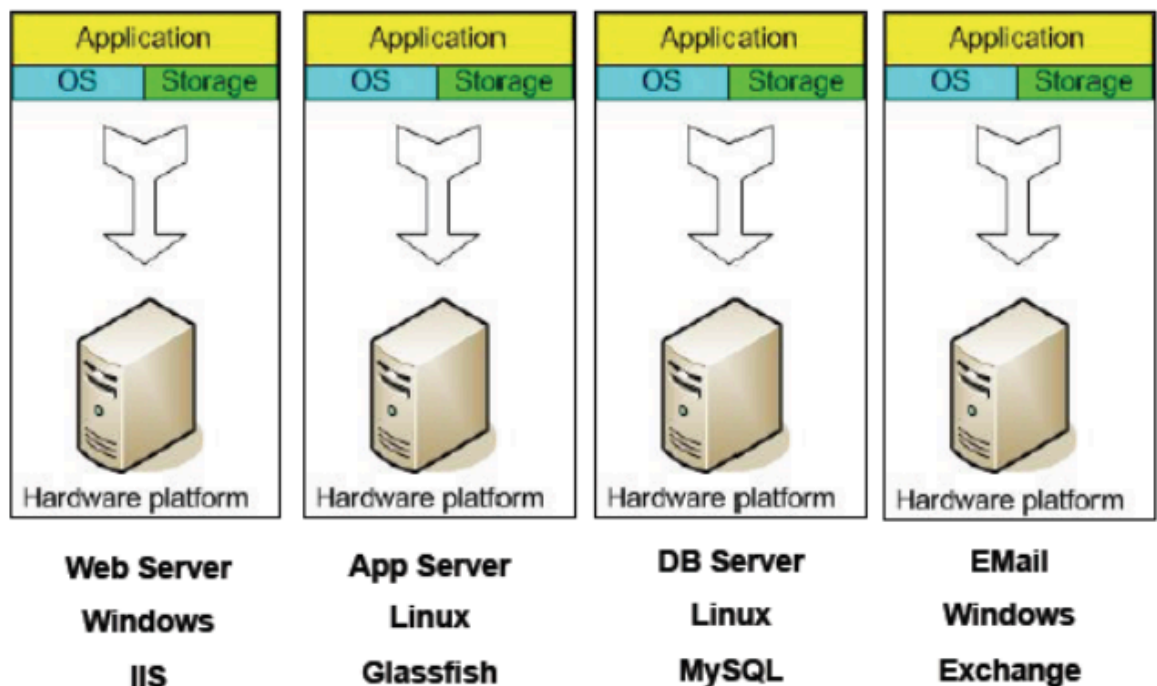
Los primeros proveedores cloud surgen de la evolución de las infraestructuras privadas de los principales ISPs

Abren a los clientes las interfaces de gestión y añaden lo necesario para implementar el modelo pay-per-use

Los clientes tienen así acceso a las mismas capacidades que antes se utilizaban sólo internamente

Virtualización

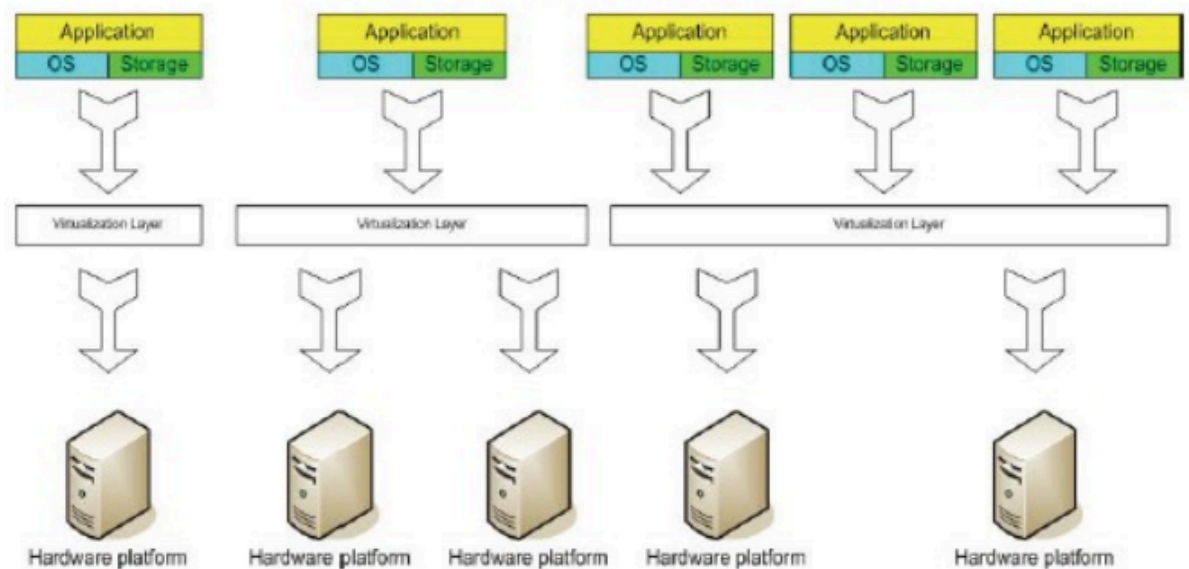
Múltiples servidores físicos



- A favor
 - Fácil de conceptualizar
 - El despliegue es bastante fácil
 - Puede ejecutar cualquier aplicación o servicio
- En contra
 - Es caro comprar y mantener el hardware
 - **No es muy escalable**
 - Es difícil replicarla
 - **Es difícil implementar la redundancia**
 - **Vulnerable a fallos hardware**
 - En situaciones de baja carga, el tiempo de CPU **se infrautiliza**

Múltiples servidores virtuales

Una capa VMM (Virtual Machine Monitor) entre el SO alojado y el hardware



- A favor
 - **Resource pooling**
 - **Alta redundancia**
 - Alta disponibilidad
 - **Despliegue rápido** de nuevos servidores
 - **Reconfigurable “en caliente”** (los servicios siguen disponibles)
 - **Optimiza la utilización** de los recursos físicos
- En contra
 - Algo más difícil de conceptualizar
 - **Ligera pérdida de rendimiento** (hay que añadir el coste de la capa de abstracción)

Cloud computing

No estamos realmente ante una nueva tecnología, sino un nuevo paradigma de uso que combina tecnologías existentes:

- SOAs
- Grid Computing
- Web Services
- Virtualización

NIST (National Institute of Standards and Technology): “Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g. networks, servers, storage, applications and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction”

ENISA (European Network and Information Security Agency): “La computación en nube es un modelo de servicio bajo demanda para la prestación de TI, a menudo basado en la virtualización y en las tecnologías informáticas distribuidas. Las arquitecturas de computación en nube poseen:

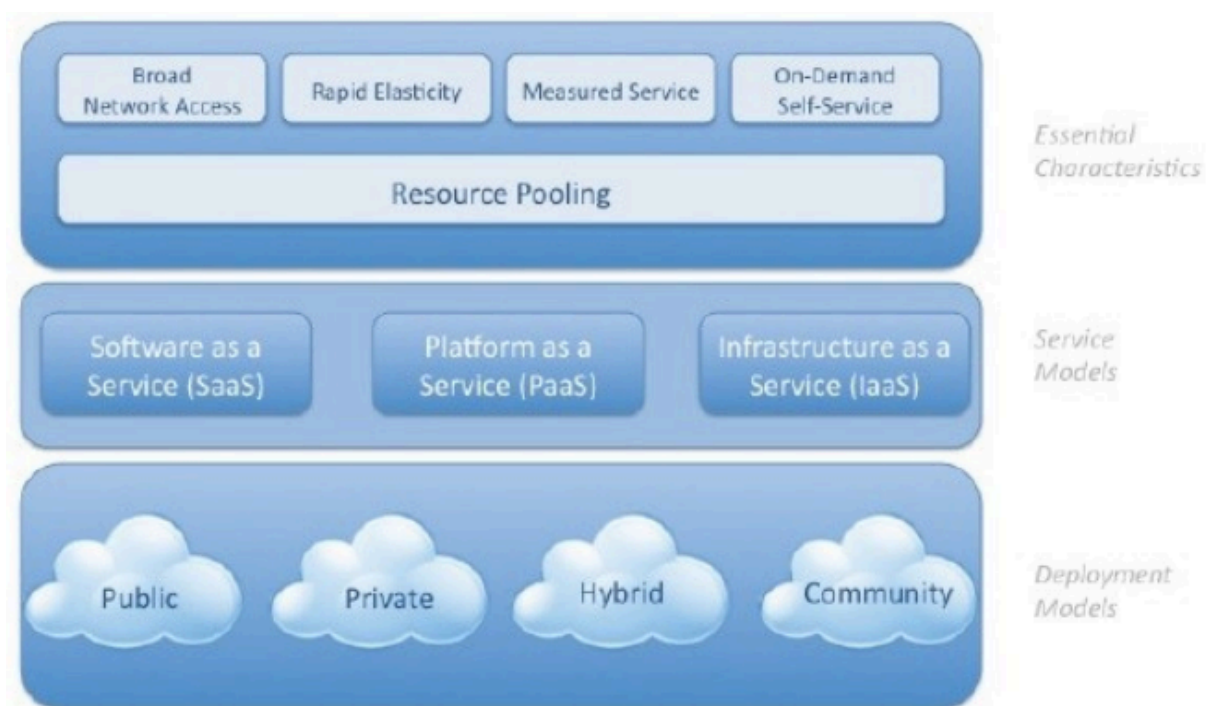
1. Recursos con un alto grado de abstracción

2. Escalabilidad y flexibilidad prácticamente instantáneas
3. Prestación casi instantánea
4. Recursos compartidos (hardware, base de datos, memoria, etc.)
5. Servicio bajo demanda que suele incluir un sistema de facturación de pago por uso
6. Gestión programática (por ejemplo, mediante la API del WS)”

Modelo 5-3-4

El modelo de Cloud del NIST es conocido popularmente como 5-3-4

- 5 Características esenciales
- 3 Modelos de servicio
- 4 Modelos de despliegue



5 características esenciales

On-demand self-service

Un consumidor puede aprovisionar unilateral y automáticamente recursos de computación según necesite

Broad network access

Las capacidades están accesibles en la red a través de mecanismos estándar que permiten el acceso desde plataformas cliente heterogéneas

Resource pooling

Los recursos de computación de los proveedores son agrupados para servir a múltiples clientes usando un modelo multi-tenant (inquilino)

Rapid elasticity

Las capacidades son aprovisionadas y liberadas rápida y elásticamente (y en algunos casos automáticamente)

Measured Service

Los sistemas cloud controlan y optimizan automáticamente el uso de los recursos proporcionando alguna capacidad de medición (habitualmente pay-per use o charge-per-use)

3 modelos de servicio

IaaS



El consumidor aprovisiona recursos de computación (p.e. CPU, almacenamiento, red, ...) en los que ejecuta su software (incluidas aplicaciones y sistemas operativos)

El consumidor no controla la infraestructura Cloud subyacente pero sí los SO, el almacenamiento, las aplicaciones desplegadas y a veces la red (firewalls)

Ejemplos



Las características comunes de un servicio IaaS son:

- Provisión de recursos bajo demanda
- Elasticidad dinámica de los recursos
- Uso eficiente de los recursos: virtualización, arquitectura multi-tenant
- Modelo de pago por uso
- Gestión más eficiente de los datacenters: beneficios de la economía de escala, estandarización, reducción de costes, eficiencia energética

Funciones de gestión

Algunas de las funciones de gestión típicas para el usuario son:

- Gestión dinámica de recursos (p.e. inicio/parada de VMs, creación/eliminación de volúmenes, ...)
- Funciones de automatización (p.e. balance automático de carga, reglas de autoescalado horizontal, ...)
- Monitorización de recursos (cpu, memoria, ancho de banda, ...)
- Gestión de consumos y costes

PaaS



El consumidor despliega aplicaciones tanto propias como adquiridas, desarrolladas usando entornos de programación soportados por el proveedor, en la infraestructura cloud de este

El consumidor no controla la infraestructura Cloud subyacente pero sí las aplicaciones desplegadas y posiblemente el entorno de despliegue

Ejemplos



Tipos de ofertas

- Desarrollo de extensiones SaaS:
 - Proveedores de SaaS proveen APIs para construir por sobre ellos (SalesForce, Facebook)
- Plataformas propietarias:
 - Algunos solo despliegan otros soportan todo el ciclo de vida del software
 - Ejemplos: Amazon Elastic BeansTalk (despliegue de aplicaciones en Amazon EC2), Microsoft Azure (plataforma .NET), Google ppEngine (APIs propias)

- Open Platform:
 - Ofertas que se basan en tecnologías abiertas para evitar el vendor lock-in
 - Son arquitecturas extensibles en las que se puede escoger el entorno de desarrollo, el lenguaje, la bd, el servidor de apps
 - Ejemplos: CloudFoundry (VMWare), OpenShift (RedHat), CumuLogic, CloudBees

¿Qué provee?

Gestión del ciclo de vida de las aplicaciones

Herramientas de desarrollo colaborativo

Herramientas para el desarrollo de UIs

Integración con WS (SOAP, REST): mashups, composición de servicios

Integración con BD SQL y NoSQL

Mecanismos de extensibilidad (plugins) para configurar nuevos frameworks

Independencia del proveedor IaaS

APIs (SOAP, REST) de gestión

Mecanismos de monitorización y medida del rendimiento

SaaS



El consumidor utiliza las aplicaciones del proveedor que son ejecutadas en una infraestructura cloud

El consumidor no controla ni la infraestructura cloud subyacente ni las capacidades de la aplicación (pero posiblemente sí puede controlar la configuración personal)

Ejemplos



Beneficios

- No hay costos de licencias
- La gestión de TI se simplifica
- Las actualizaciones son automáticas y constantes
- Ubicuidad (accesible desde cualquier dispositivo)
- Integración a través de APIs

Desventajas

- Dependencia de la red
- Adaptabilidad
- Aspectos legales y de seguridad
- Dependencia del vendedor
- Costos: para PyMEs es ventajoso

Beneficios SaaS vs on-premise

Análisis del ROI de implantar un ERP con horizonte

Costes		On-premise	SaaS
Software ERP	\$1.000.000	(mensual) \$10.000	
Hardware	\$300.000		\$0
Instalación/consultoría	\$500.000		\$0
Soporte interno	\$0	(mensual) \$10.000	
Operación (anual)	\$600.000		\$0
Total	\$3.600.000		\$780.000
año 1		año 2	año 3
On-premise	\$2.400.000	\$600.000	\$600.000
SaaS	\$240.000	\$240.000	\$300.000

SaaS vs ASP (Application Service Provider)

Muchas similitudes:

- Aplicaciones bajo demanda
- Acceso ubicuo
- Pago por uso
- El cliente no gestiona ni controla la infraestructura subyacente
- Configuraciones personalizadas
- Reducción de costes de instalación y operación

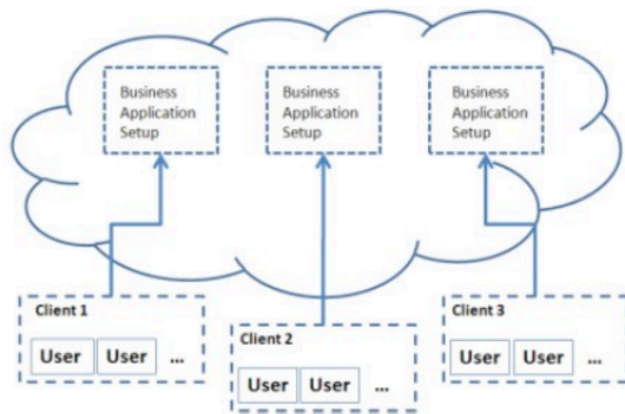
La diferencia principal está en la infraestructura del proveedor (single-tenant vs multi-tenant)

La importancia del aaS en SaaS

Interfaces WS abiertas: REST, SOAP, JSON, etc.

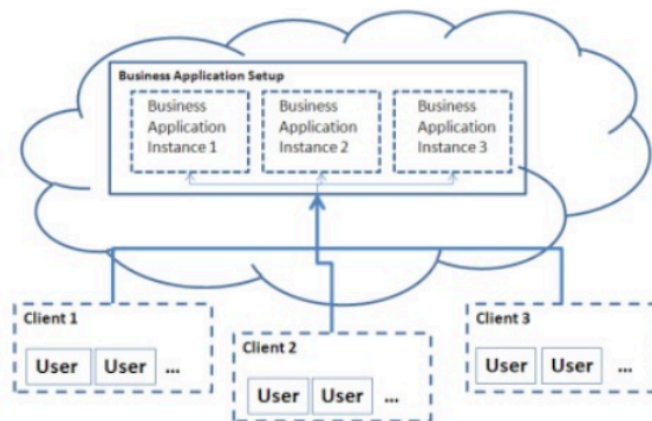
Facilidad de integración, mashup, orquestación, etc.

Desde el punto de vista del usuario es difícil percibir la diferencia



ASP: single-tenant

- Cada cliente tiene su propia instalación personalizada que no se comparte y puede incluir la configuración del software, BD, servidores, etc.
- Los entornos ASP son más heterogéneos para dar soporte a las diferencias entre clientes y a los "casos especiales"
- El mantenimiento es, por tanto, más costoso
- La escalabilidad es limitada y difícil de implementar.



SaaS: multi-tenant

- Todos los clientes comparten la misma infraestructura (servidores, BD, aplicaciones, ...) al tiempo que se preserva la separación entre ellos
- Los entornos SaaS son más homogéneos
- El mantenimiento es, por tanto, más barato y la escalabilidad más fácil de implementar.
- Tiene la contrapartida de ser menos flexible para adaptarse a las peculiaridades de los clientes

Variantes aaS

- HaaS: Hardware-aaS
- BPaaS: Business Process-aaS
- IDaaS: Identity-aaS
- EaaS: Entertainment-aaS
- BDaaS: Bases de datos-aaS

4 modelos de despliegue

Cloud público

Infraestructura para uso del público o empresas en general

Propiedad del proveedor (p.e. empresa, universidad, organismo estatal, ...)

Estos clouds tienen una interface para la gestión interna de la infraestructura virtualizada y exponen otra externa para la gestión de los recursos de los clientes



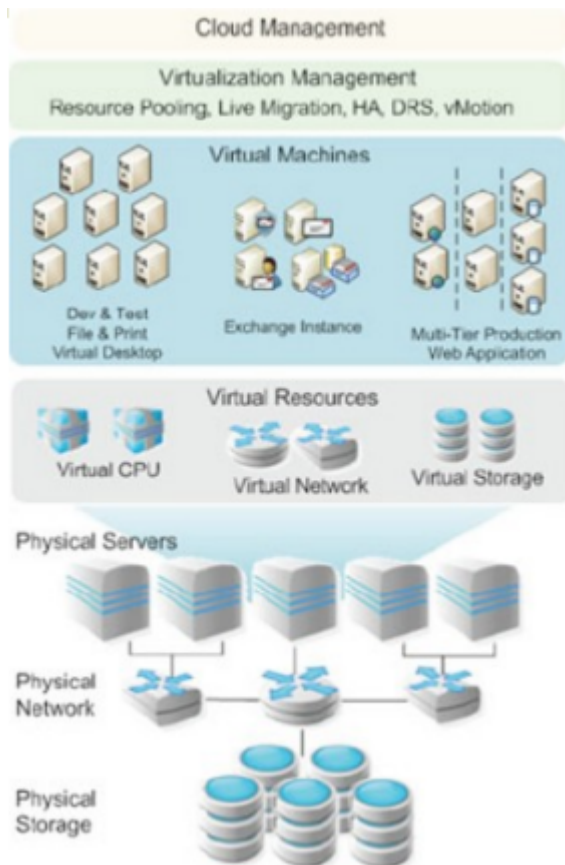
Cloud privado

Para uso exclusivo de una organización con múltiples clientes (p.e. departamentos de una empresa)

Puede ser de su propiedad (on-premise), alquilarlo (off-premise) o una combinación de ambas

Permite una gestión flexible y ágil de los recursos de la organización

Los principales proveedores públicos de IaaS comenzaron creando clouds privados para mejorar la gestión de sus datacenters

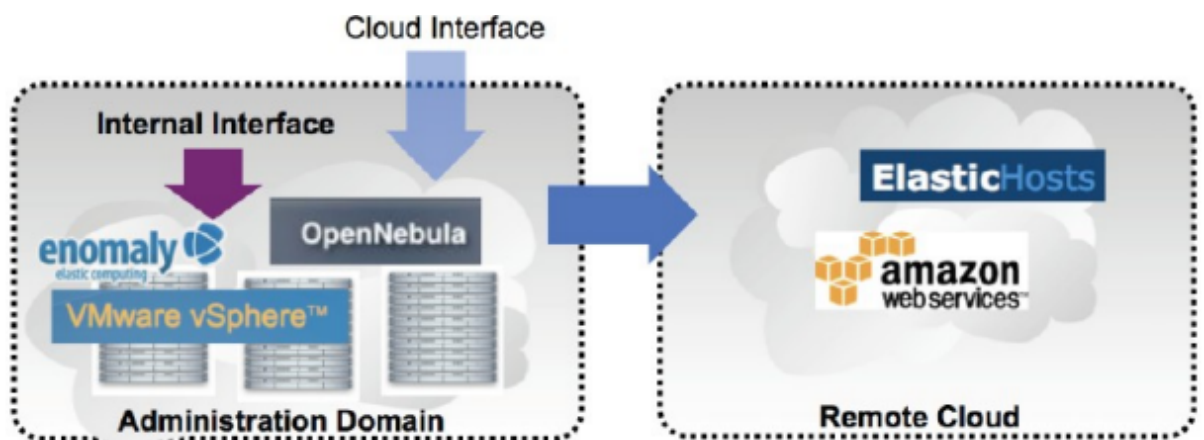


Cloud híbrido

Combinación de cloud privado y público

Permite gestionar los picos de carga obteniendo recursos del cloud público

El uso de un cloud público es totalmente transparente para los usuarios del cloud privado



Cloud comunitario

Infraestructura para uso de una comunidad de organizaciones que comparten intereses

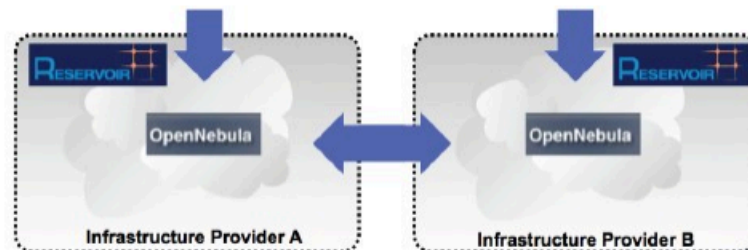
Cada componente conserva su autonomía

La relación entre componentes se hace mediante tecnologías (propietarias o estandarizadas) que permitan la portabilidad de datos y aplicaciones

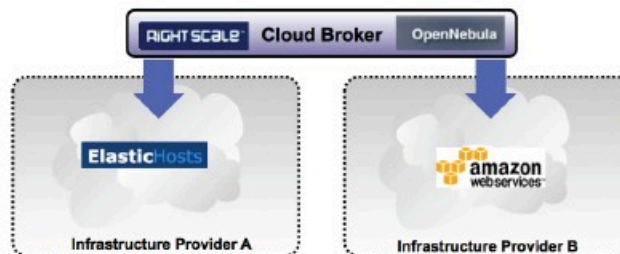
Puede ser propiedad de una o varias de las organizaciones, ser alquilado o una combinación de ambas

Otros modelos de despliegue

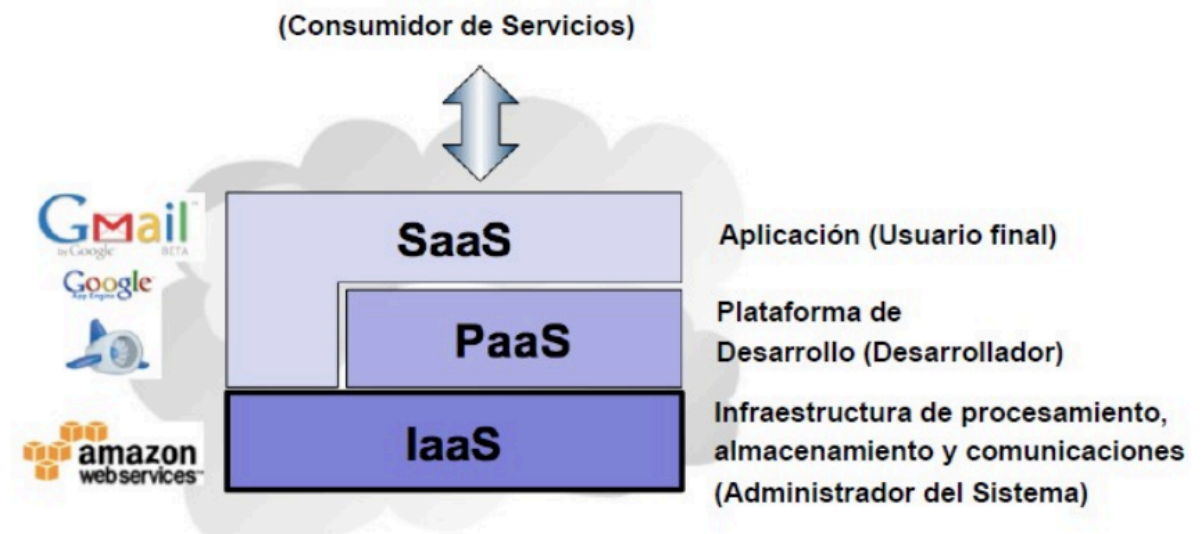
Federación de clouds (requiere estándares)



Cloud brokering (uso conjunto de varios proveedores)



Resumen



Con IaaS contratamos: computación, almacenamiento y red

	Amazon	Microsoft Azure	Google
Computación	EC2 Auto scaling ELB	Virtual Machines Traffic Manager	Compute Engine
Almacenamiento	S3 EBS DynamoDB Glacier RDS	Storage Backup SQL Database	Cloud Storage Cloud SQL
Red	Elastic IP VPC Route 53 Direct Connect	Virtual Network	

Estándares en el cloud

Hay numerosas iniciativas promoviendo estándares y su adopción: NIST, CSA (Cloud Security Alliance), Open Data Center Alliance, OCI (Open Cloud Initiative), Open Cloud Consortium, etc.

Se han identificado varias áreas en las que es necesario intensificar los esfuerzos de estandarización:

- Portabilidad de VMs e imágenes
- Acceso y migración de datos
- Seguridad y gestión de la identidad
- Orquestación de procesos
- Monitorización y administración
- SLAs

Estándares IaaS

OVF (Open Virtualization Format)

Estándar para el empaquetamiento y la distribución de VMs

Define un formato abierto, extensible, eficiente e independiente de la plataforma

Propuesta apoyada por los principales actores en virtualización (VMware, Citrix, Microsoft, ...)

CIMI (Cloud Infrastructure Management Interface)

Interfaz para la gestión de infraestructuras IaaS

Alternativa al estándar de facto: Amazon EC2

“Contendiente” del estándar OCCl

Su enfoque es hacia la gestión del ciclo de vida de los recursos proporcionados por los proveedores IaaS a sus clientes

Define un modelo para los recursos (computación, almacenamiento, comunicaciones) que puede ser mapeado a la oferta de cualquier proveedor de infraestructura Cloud

Aún no hay muchas implementaciones: Apache DeltaCloud, OW2 Sirocco

OCCl (Open Cloud Computing Interface)

Comenzó siendo una API REST para la gestión de infraestructura IaaS

Alternativa al estándar de facto: Amazon EC2

Propuesta apoyada por muchos proveedores IaaS y proyectos y plataformas Abiertos (Rackspace, Oracle, GoGrid, Cisco, Flexiscale, ElasticHosts, SLA@SOI, RESERVOIR, OpenStack, OpenNebula, etc.

Numerosas implementaciones: OpenNebula, OpenStack, Eucaliptus, etc.

CDMI (Cloud Data Management Interface)

Interfaz para el manejo de diferentes tipos de almacenamiento en el cloud

Soporta: contenedores y objetos, colas, tablas

Permite asociar metadatos a contenedores y objetos

Apoyado por muchas empresas importantes: DELL, IBM, Oracle, Intel, Fujitsu, Microsoft, VMware, etc.

Adoptado en el 2012 como estándar ISO/IEC 17826:2012

Estándares PaaS

CAMP (Cloud Application Management for Platforms)

Interfaz estándar para el acceso a los servicios de proveedores PaaS independiente de la plataforma, framework y lenguaje de desarrollo utilizados

Define los artefactos y APIs necesarios para crear, ejecutar, administrar, monitorear y “patchear” aplicaciones en el Cloud TOSCA (Topology and Orchestration Specification for Cloud Applications)

Estándar para la descripción de aplicaciones y servicios IT complejos a ser ejecutados sobre infraestructuras Cloud

Su enfoque es hacia la portabilidad de aplicaciones y servicios con independencia del proveedor Cloud utilizado

Define un metamodelo declarativo de Service Templates, en los que se describe:

- La topología (o estructura) del servicio
- Los planes u operaciones que permiten iniciarlo, terminarlo y gestionar su ciclo de vida

Beneficios:

- Despliegue de aplicaciones en cualquier proveedor manteniendo los requisitos de la aplicación: seguridad, compatibilidad, etc.
- Facilita la migración de aplicaciones existentes hacia entornos Cloud
- Abre la posibilidad a aplicaciones multiproveedor dinámicas
- Creación de un mercado de servicios IT

Seguridad en cloud computing

Amenazas, desafíos y guías asociadas a la infraestructura de IT en la nube a nivel de:

- La red
- Los host
- Las aplicaciones

Generalmente se asocia la seguridad a IaaS, pero también debe considerarse a nivel de SaaS y PaaS ya que algunas son propias del modelo de servicio

Infraestructura de seguridad a nivel de red

Es importante distinguir entre cloud privado y público

Cloud privado: no hay consideraciones adicionales, ataques, vulnerabilidades o cambios específicos de la topología que la seguridad personal haya ya considerado

Cloud público: las consideraciones de seguridad afectan la topología de la red debiendo revisarse 4 aspectos:

- Asegurar la confidencialidad e integridad de los datos transmitidos desde y hacia el proveedor de cloud
- Asegurar un protocolo de control de acceso propio
- Asegurar la disponibilidad de los recursos expuestos a Internet
- Reemplazar el modelo establecido de zonas, niveles y dominios

Asegurar la confidencialidad e integridad de los datos transmitidos desde y hacia el proveedor de cloud

Muchos datos y recursos anteriormente confiados a una red privada ahora están expuestos a Internet y a una red compartida que pertenece a un proveedor externo

Un ejemplo de ello fue una falla en el algoritmo de firma digital que mostró la vulnerabilidad de AWS (Amazon Web Services) en 2008

Una consulta a SimpleDB sobre http mostró la vulnerabilidad del sistema. Si bien usar HTTPS en lugar de HTTP puede mitigar los riesgos de seguridad, permitir

que los usuarios utilicen HTTP enfrentó a los usuarios con un riesgo sobre la integridad de los datos

Asegurar un protocolo de control de acceso propio

Los recursos están expuestos a Internet

Las organizaciones ven limitadas sus capacidades de revisión de logs

Problemas del uso del mecanismo de reuso de direcciones IP y la desincronización de la reasignación y su actualización en los DNS

Si bien hay productos para mitigar el riesgo de reuso de IPs, queda en manos del proveedor de cloud adoptarlos

Asegurar la disponibilidad de los recursos expuestos a Internet

La exposición de recursos a Internet ha ido aumentando y se potencia en el cloud:

- Secuestro de Prefijo:
 - En inglés prefix hijacking de los BGP (Border Gateway Protocol, protocolo de ruteo entre dominio usado en Internet)
 - El secuestro de prefijo implica anunciar un espacio de direcciones de un sistema autónomo que pertenece a alguien más sin su permiso. Tales anuncios pueden ser errores de configuración o también por ataques. Ejemplo: youtube fuera de servicios por 4 horas en 2008
- Ataque a DNS:
 - Es una práctica muy habitual y que se ve incrementada su complejidad en el cloud donde se reduce ampliamente la efectividad de la técnica de Split Horizon en DNS (provee distintas respuestas autorizadas a una misma consulta para asegurar la veracidad de los datos retornados por el DNS)
- DoS:
 - Es el intento de quitar de servicio un host logrado en general a través de la generación de requerimientos falsos con el objetivo de sobrecargar el sistema evitando que atienda los requerimientos legítimos
 - Si bien esto es habitual aún sin una infraestructura cloud, en el caso de cloud se ve incrementado no solo sobre los recursos

expuestos a Internet sino dentro mismo de su esquema interno

- Asimismo, el uso de IaaS hace que la red interna del proveedor de cloud sea un recurso compartido que cae en riesgo de ser denegado como servicio. En este caso es responsabilidad de cada cliente proteger sus instancias

Reemplazar el modelo establecido de zonas, niveles y dominios

El modelo de aislamiento basado en zonas y niveles ya no es posible en cloud públicos tanto para IaaS como para PaaS

Durante mucho tiempo la seguridad de la red se apoyó en la definición de zonas tales como intranet vs extranet o entornos de producción vs entornos de desarrollo, de modo que separando el tráfico se garantizaba la seguridad y se basaba en la exclusión

El modelo de zonas y niveles ha sido reemplazado en el cloud computing público por “seguridad de grupos”, “seguridad de dominios” y “virtualización”, que poseen una separación lógica entre niveles pero son menos precisos y aportan menos protección que el esquema anterior

¿Cómo podemos mitigar los factores de riesgo?

Primero notemos que los riesgos a nivel de red no son consecuencia del modelo de servicio, o sea del tipo deaaS sino que depende de si la organización usa un cloud público, privado o híbrido, es decir del modelo de despliegue

Si la organización puede afrontar el costo de un cloud privado, los riesgos se verán decrementados

La confidencialidad puede incrementarse utilizando encriptación con implementaciones robustas de cifrado de datos en tránsito

Los problemas de disponibilidad a nivel de red son más difíciles de mitigar, aun cuando su red privada no está montada sobre un proveedor de cloud externo, ya que igualmente conlleva riesgos a nivel de red. Un cloud público no enfrenta problemas mayores que cualquier seguridad de red

Infraestructura de seguridad a nivel de host

Para revisar aspectos de seguridad a nivel de host, tanto del host en sí, como de sus accesos, debemos considerar tanto el modelo de servicio (aaS) como el modelo de despliegue (público o privado)

Las nuevas amenazas a los host se relacionan con la virtualización (escapes de la VM, fallas en el sistema de configuración, amenazas por el débil control de acceso a los hipervisores)

La naturaleza dinámica o elasticidad en el aprovisionamiento trae también consecuencias ya que la fugacidad de las instancias de VM no permite manejar las vulnerabilidades y actualizar rápidamente las correcciones o patches

El aprovechamiento del poder de miles de nodos con un sistema operativo homogéneo, hace que las amenazas se amplifiquen y se multipliquen más rápidamente

Esto da cuenta de los límites de confianza y responsabilidad que estamos delegando en los proveedores de cloud

Seguridad a nivel de host en SaaS y PaaS

En el contexto de SaaS y PaaS la seguridad de host es opaca a los clientes y la seguridad es total responsabilidad del proveedor

Generalmente el proveedor debe ofrecer garantía de NDA (Non-Disclosure Agreement o acuerdo de no divulgación) y asegurar controles preventivos y detectivos apropiados que se ajusten a la SysTrust - ISO 27002

Seguridad a nivel de host en IaaS

Seguridad del software de virtualización

Los proveedores cloud gestionan el software de virtualización que se aloja sobre el hardware, los clientes no tienen visibilidad sobre ellos ni acceden a ese software

El hardware y SO de virtualización permiten compartir recursos de hardware a través de múltiples VMs sin interferir unas con otras y cada uno ejecuta sus aplicaciones al mismo tiempo en una única computadora

La integridad y disponibilidad del hipervisor son fundamentales y son la clave para garantizar la integridad y disponibilidad del cloud público construido sobre un ambiente virtualizado

Un hipervisor vulnerable podría exponer a todos los usuarios a intervenciones maliciosas

Dado que el software de virtualización en cloud públicos son propietarios y de código cerrado (si bien algunos emplean software open source como Xen) el código fuente usado por el proveedor no está disponible para ser examinado

Seguridad del servidor virtual

Los clientes de IaaS tienen acceso completo a las VM que son alojadas y aisladas unas de otras por la tecnología de hipervisores, por lo tanto los clientes son responsables de la seguridad y su gestión de la VM

Un IaaS público como EC2 de Amazon, ofrece una API basada en WS para la gestión de aprovisionamiento, desmantelamiento y replicación de servidores virtuales

Estas funciones se orquestan ofreciendo un ciclo de vida dinámico que puede ser complejo si los servidores virtuales no están automatizados con procedimientos adecuados

Desde la perspectiva de la superficie, el servidor virtual (Windows, Solaris o Linux) puede ser accesible a cualquiera desde Internet. Por ello los proveedores de cloud bloquean todos los puertos de acceso y recomiendan usar el puerto 22 (Secure Shell) para administrar las instancias de los servidores virtuales

Algunas amenazas de seguridad en IaaS público incluyen:

- Robo de claves usadas para acceder a la administración
- Ataques a servicios de escucha vulnerables sobre puertos estándar (como FTP)
- Secuestro de cuentas no securizadas adecuadamente (password débiles)
- Ataques al sistema no securizados por firewalls
- Despliegue de troyanos embebidos en el software de la VM o en su imagen

Infraestructura de seguridad a nivel de aplicación

La seguridad en aplicaciones y software es un punto crítico en el cual las empresas que producen este software invierten parte de su tiempo

Desarrollar e implementar aplicaciones en el cloud requiere revisar las prácticas habituales

El espectro de seguridad va desde simples aplicaciones stand-alone a sofisticados sistemas de e-commerce, pasando por las aplicaciones web como CMS, wikis, foros e incluso redes sociales

Según SANS hasta 2007 eran pocos los ataques a sitios web vulnerables, pero luego, el incremento de XSS y otros ataques demostraron que la programación web puede ser usada para penetrar en las organizaciones

Por eso nos centramos en la discusión de la seguridad en aplicaciones web en el cloud accedidas por browsers desde cualquier computadora conectada a Internet

Denegación de sostenibilidad

Se debe tomar conciencia a nivel de aplicación de los ataques de denegación de sostenibilidad inspeccionando un alto volumen de recargas de páginas, los requerimientos de los WS por HTTP o HTTPS

Dado que este tráfico malicioso viene incluido en el tráfico válido es de difícil detección y además de alterar el servicio de cloud, también incrementa las tarifas de pago por uso de la red

Por su parte la elección de cloud por parte de pequeñas y medianas empresas que no pueden asumir los costos de inversión, mantienen las barreras bajas de seguridad

Seguridad de usuarios finales

Las medidas de seguridad que adopten los usuarios finales como uso de antivirus, anti-malware, firewalls personales y parches de seguridad, contribuyen a mantener la seguridad y una navegación segura que impacta en la seguridad de las aplicaciones siendo que los browsers se han convertido en sistemas operativos ubicuos

Más aún, para asegurar una seguridad punta a punta en el cloud, es fundamental mantener una buena higiene de los navegadores

Seguridad de aplicaciones SaaS

En un entorno SaaS el proveedor del servicio maneja la suite entera de la aplicación que se dispone a los usuarios, de modo que la seguridad también es su responsabilidad

El cliente suele ser responsable de la seguridad operacional incluyendo la gestión de usuarios y accesos que otorga el proveedor

Se suele realizar testing de penetración y poner atención extra a las características de autenticación y control de acceso

Seguridad de aplicaciones PaaS

El uso de PaaS es aún muy incipiente y la recomendación es que las organizaciones que lo vayan a incorporar usen las mismas consideraciones que al adquirir un paquete de desarrollo de software tradicional

Conclusión de seguridad

Hemos revisado la seguridad en el cloud desde el aspecto de la red, de los host y de las aplicaciones

A nivel de red, si bien es un desafío para cloud computing, ninguno de ellos es intrínseco del cloud en sí mismo. Todos los aspectos de seguridad en la red están exacerbados por el cloud, pero no son causados por él

En cuanto a la seguridad de host, se requiere incrementar la seguridad perimetral y dar entornos de virtualización seguros. Otra vez los aspectos de seguridad se ven exacerbados pero no son causados por el cloud

A nivel de aplicación se sugiere utilizar buenas prácticas de seguridad en el desarrollo y tomar las responsabilidades pertinentes de cada uno de los actores

Clase 6 - Monitoreo de procesos de negocio

Business Process Management

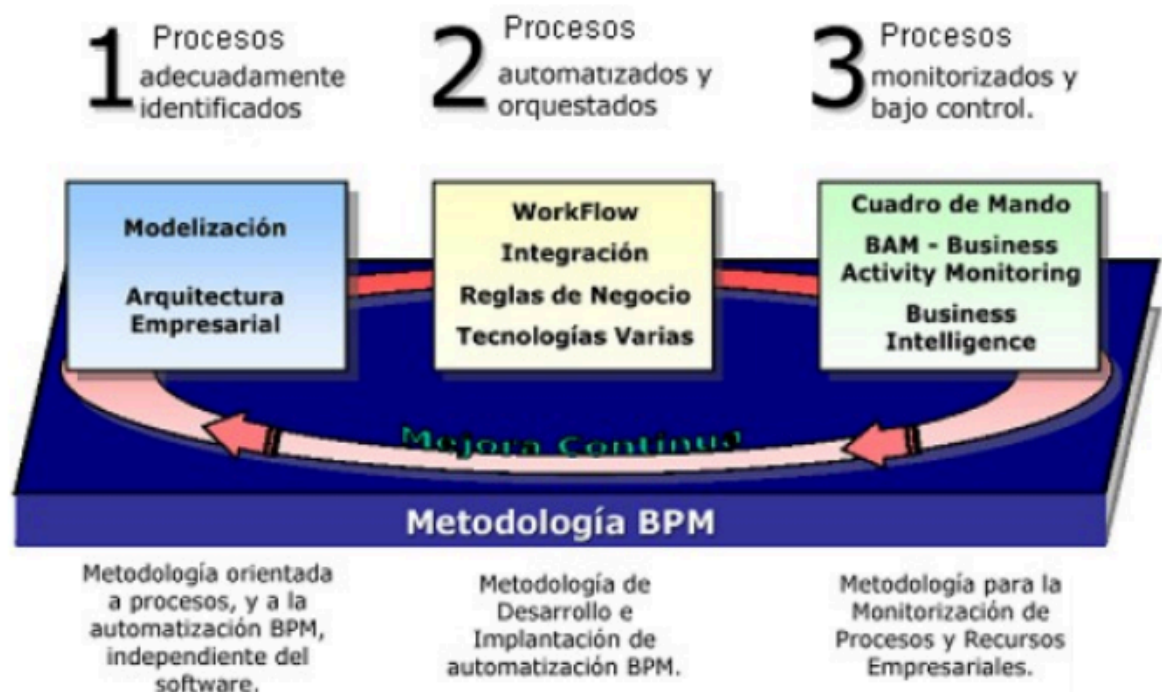
Producto: a un conjunto de actividades realizadas para obtenerlo

Proceso: ordenamiento específico de actividades en un tiempo y lugar determinado, con un principio y un fin, con entradas y salidas y para lograr un determinado objetivo

BPM incluye conceptos, métodos y técnicas para soportar el diseño, administración, configuración, representación y análisis de los procesos de negocio

Etapas de trabajo con la metodología BPM

1. Procesos adecuadamente identificados
2. Procesos automatizados y orquestados
3. Procesos monitorizados y bajo control



Analizar, en función de los indicadores, el comportamiento del proceso según sus objetivos

Elementos de un proceso

- Mapa y diagrama de procesos
- Actores y propietario
- Reglas de negocio
- Indicadores (eficacia, eficiencia, rastros de tiempo)
- Subprocesos o actividades clave

BPMS

Poseen herramientas de Análisis y Business Intelligence: estudian la información resultante de la ejecución del proceso en tiempo real

Facilitan la gestión de los procesos de negocio permitiendo, entre otras cosas, el monitoreo en tiempo real (BAM)

BAM (Business Activity Monitoring)

Objetivo: recopilación, organización, análisis y visualización de datos obtenidos en tiempo real, acerca de las actividades ejecutadas en un proceso de negocio

Beneficia la toma rápida de decisiones

Permite, identificar y resolver problemas durante la ejecución del proceso

Bajo un BPMS, la monitorización de los procesos permite obtener en tiempo real información de los indicadores clave que se hayan definido con objeto de contar con un soporte para su mejora continua

Se puede utilizar para simulación en la fase de definición:

- Evaluar distintos escenarios posibles
- Calcular indicadores
- Detectar cuellos de botella, tiempos de espera, etc.

Introduce nuevas estrategias de negocio, reducir costos de operación, mejorar desempeño de los procesos y otras áreas tangibles de interés para la administración

Es la convergencia de diferentes clases de tecnologías, incluyendo:

- Inteligencia de Negocios Operacional
- Integración de aplicaciones en tiempo real
- Administración del negocio

Componentes de BAM

- Eventos del negocio
- Actividades del negocio
- Métricas → se realizan sobre una actividad
- Indicadores clave de desempeño (KPI) → son a nivel de proceso

Funcionalidad de BAM

- Captura instantánea de datos
- Visualización de datos
- Notificaciones automáticas
- Generación de reportes

Métricas de un proceso de negocio

La optimización de procesos de negocio es una tarea compleja que requiere un buen conocimiento de los mismos dentro de la organización. Para ello se deben llevar métricas de los mismos, que permiten medir una instancia del proceso

KPIs

Key Performance Indicators

Ligados a la/s estrategia/s de la organización

Permiten realizar la medición de los procesos de la organización

Mueven a la organización en la dirección correcta para alcanzar sus metas

Son a nivel de proceso, de todas las instancias que están corriendo. Se pueden ver como una función de agregación de alguna métrica

Se deben elaborar KPIs determinando la información a mostrar y que resulte útil para mejorar la performance de los procesos, así como el diseño de métricas y alertas

Un KPI es una parte clave de un objetivo medible

Por ej.: costo en producir una pieza

Todo proceso puede medirse, al menos, en cuatro dimensiones:

- Costo
- Calidad
- Tiempo
- Satisfacción del cliente (en el sentido de estar en sincronismo con los requisitos exigidos)

Cuando se definen KPIs se aplica el acrónimo SMART, ya que los KPIs tienen que ser:

- Specific: ¿indica claramente qué queremos lograr con el proyecto?
- Measurable: ¿Muestra el progreso hacia el resultado esperado?
- Attainable: ¿es alcanzable con los recursos disponibles?
- Relevant: ¿contribuye al éxito del proyecto?
- Time-bound: ¿es posible alcanzarlo en el tiempo indicado?

Specific

Para poder acotarlo claramente debemos responder a 5 preguntas clave:

- ¿Qué quiero conseguir?
- ¿Qué beneficios o motivos específicos tengo para ello?
- ¿Quién lo va a conseguir
- ¿Dónde lo voy a hacer?
- ¿Qué necesito para hacerlo?

Measurable

Tenemos que poder medir nuestro progreso y para ello debemos poder responder a las siguientes preguntas:

- ¿Qué quiero alcanzar?
- ¿Cómo voy a medir mi progreso? → Seguiré un recuento, diario, semanal, mensual, etc.

Attainable

Tienen que ser alcanzables con los recursos que disponemos:

- ¿Son asequibles mis KPIs?
- ¿Lo podremos hacer con los recursos de los que disponemos?

Relevant

Identificar los retos que se nos plantean, y analizar si son o no importantes:

- ¿Este KPI es importante para mi negocio? → ¿Del 1 al 10 dónde lo situaría?
- En un futuro, ¿encajará este KPI dentro de la perspectiva de mi plan de negocio?

Time-bound

Definir y seguir un tiempo para los KPIs:

- ¿Cuándo quiero tener cumplidos mis KPIs?
- ¿Es posible realizarlo en ese tiempo?
- ¿Tengo definidas las estrategias para cumplir mis KPIs?

Conclusión KPIs

Criterios de calidad propuestos:

- Selectividad o importancia
- Simplicidad
- Representatividad
- Indagación
- Comparabilidad
- Estabilidad
- Relación costo - efectividad
- Sencillo y Claro
- Fácil y rápido de elaborar
- Pertinente
- Adaptado

Clase 7 - SQL y NoSQL

En general para la gestión de un sistema de información es necesario la normalización de sus datos

Lo ideal es que la información de una organización esté estructurada de un modo conocido para poder manejarla, almacenarla, recuperarla

Para este proceso se definen modelos de datos

Un sistema es mucho más que datos

Surgen los frameworks con el propósito de normalizar y estructurar el código del sistema

En general, divide la aplicación en tres capas:

- La lógica de presentación
- La Lógica de datos
- La lógica de dominio o de negocio

Gestión de datos

Se debe tener en cuenta el propósito de la aplicación:

- Bases de datos SQL
- Bases de datos NoSQL (Not Only SQL)

La diferencia fundamental: resuelven escenarios completamente distintos. El ideal SQL no lo es para NoSQL

Diferencias SQL y NoSQL

SQL	NoSQL
Permite combinar tablas y relacionar información	No lo permite o muy limitado
Facilita distribuir Bases de Datos Relacionadas	Distribuir grandes cantidades de información
Gestión de datos y sus relaciones	No existen estos conceptos
Datos consistentes sin dar posibilidad de error	Las estructuras de datos manejadas son variables
Sistemas de gestión de información operativa	Análisis de grandes cantidades de datos en modo lectura
Auditoria y trazabilidad de los datos registrados	Captura y procesamiento de eventos
Claridad conceptual en la relación de los datos que se deben representar	Tiendas online con motores de inteligencia complejos

SQL

SQL es una herramienta para organizar, gestionar y recuperar datos almacenados en una BD relacional

Tiene diferentes roles, es un lenguaje:

- De consulta interactivo

- De programación de bases de datos
- De definición y administración de datos
- De bases de datos distribuidas

Ayuda a proteger los datos en un ambiente multiusuario (control de acceso)

Garantiza la integridad y consistencia de datos en un ambiente multiusuario

¿Qué hace un servidor SQL?

Controla y ejecuta comandos SQL

Provee una visión lógica y física de los datos

Genera planes optimizados de ejecución de los comandos SQL

Maneja la recuperación, concurrencia, seguridad y consistencia de la base de datos

Controlan la ejecución de transacciones

Obtienen y liberan lockeos durante la ejecución de la transacción en curso

Arquitectura de los servidores SQL

La arquitectura se puede definir desde dos puntos de vista:

- Desde el punto de vista del almacenamiento: centralizado o distribuido
- Desde el punto de vista del procesamiento: arquitectura multithreaded, tiene su propio scheduler independiente del SO. Por ejemplo: Sybase y SQL Server

Store procedures

Es una colección de sentencias SQL y lógica procedural

Es un objeto más de la base de datos y es almacenado en el catálogo

Acepta parámetros de entrada

Se gana enormemente en eficiencia y reducción de tráfico en la red

Triggers

Un trigger es una acción especial definida por el usuario (usualmente en la forma de SP) que son automáticamente invocadas cuando ocurre un evento relacionado con los datos de la base de datos

Se habilitan implícitamente por las operaciones de DELETE, INSERT y UPDATE sobre las tablas para las cuales están definidos

Existen dos modelos de ejecución de los triggers:

- After (después de la operación para la que fue definido)
- Before (antes de la operación para la que fue definido)

Según el modelo, se altera el orden de ejecución de las restricciones de integridad

Reglas

Una regla es un caso especial de trigger que se usa para chequeos simples sobre los datos

SQL estático y dinámico

El SQL estático son sentencias definidas en el código y compiladas en el momento de su definición

Si los objetos que referencia no existen, no compila

El SQL dinámico es creado en tiempo de ejecución y ofrece más flexibilidad a expensas de velocidad de ejecución

SQL incorporado y programado

SQL incorporado: las sentencias SQL están incorporadas al código fuente del programa, entremezcladas con las otras sentencias del lenguaje de programación

SQL programado: las sentencias SQL se almacenan como objetos de la base de datos (triggers, stored procedure, reglas)

	Procedimientos almacenados	SQL embebido estático	SQL embebido dinámico
Nombres para las funciones	SI	NO	NO
Funciones compartidas	SI	NO	NO
Parametros de E/S	SI	NO	NO
Catalogado	SI	SI	NO
Logica procedural	En el objeto	Externa	Externa
Flexibilidad	Baja	Baja	Alta
Nivel de abstraccion	Alto	Bajo	Bajo
Estandarizacion	NO	SI	SI
Performance	Rapido	Mediano	Lento
Trafico	1 Request/Reply por varios comandos SQL	1 Request/Reply por cada comando SQL	1 Request/Reply por cada comando SQL

Acceso concurrente

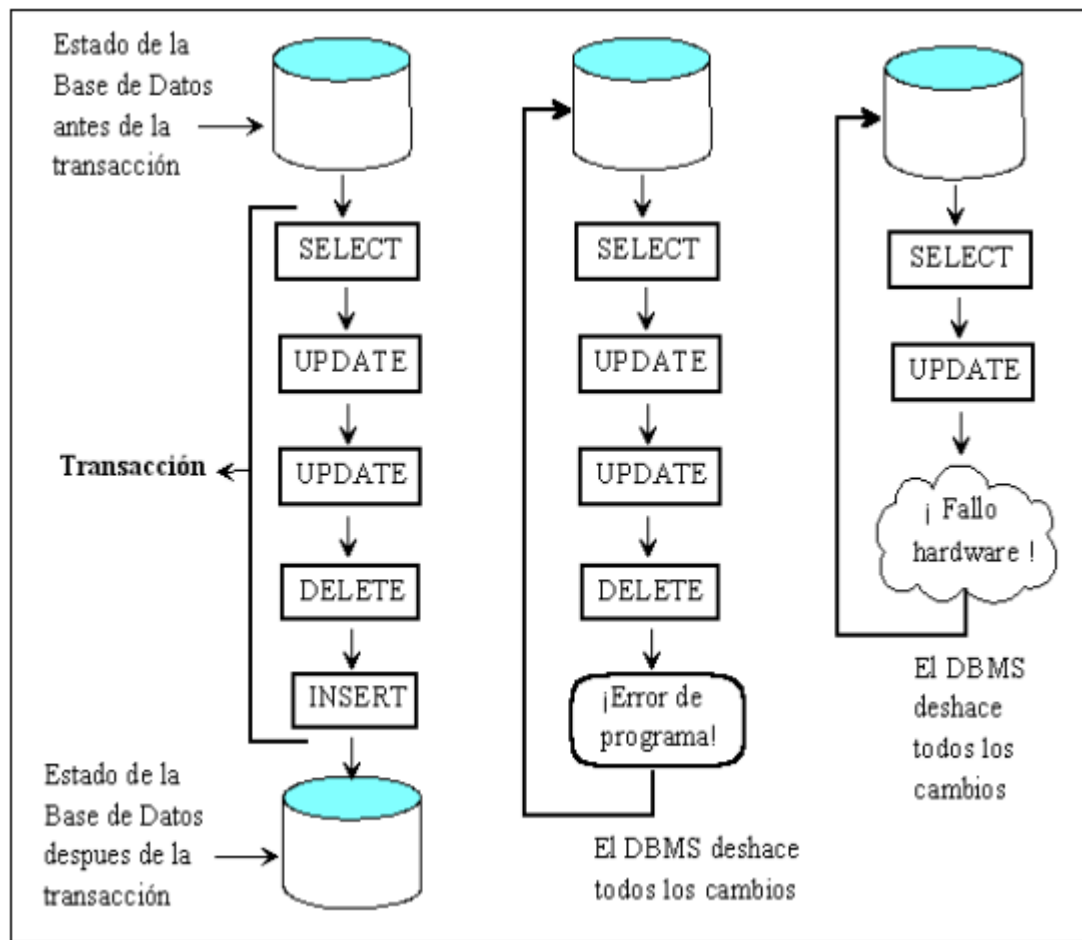
Ayuda a proteger los datos en un ambiente multiusuario (control de acceso), garantizando la integridad y consistencia de datos

Lo hace a través de los siguientes servicios:

- Control de ejecución de transacciones
- Obtener y liberar lockeos durante la ejecución de la transacción en curso
- Proteger la base de datos de accesos no autorizados

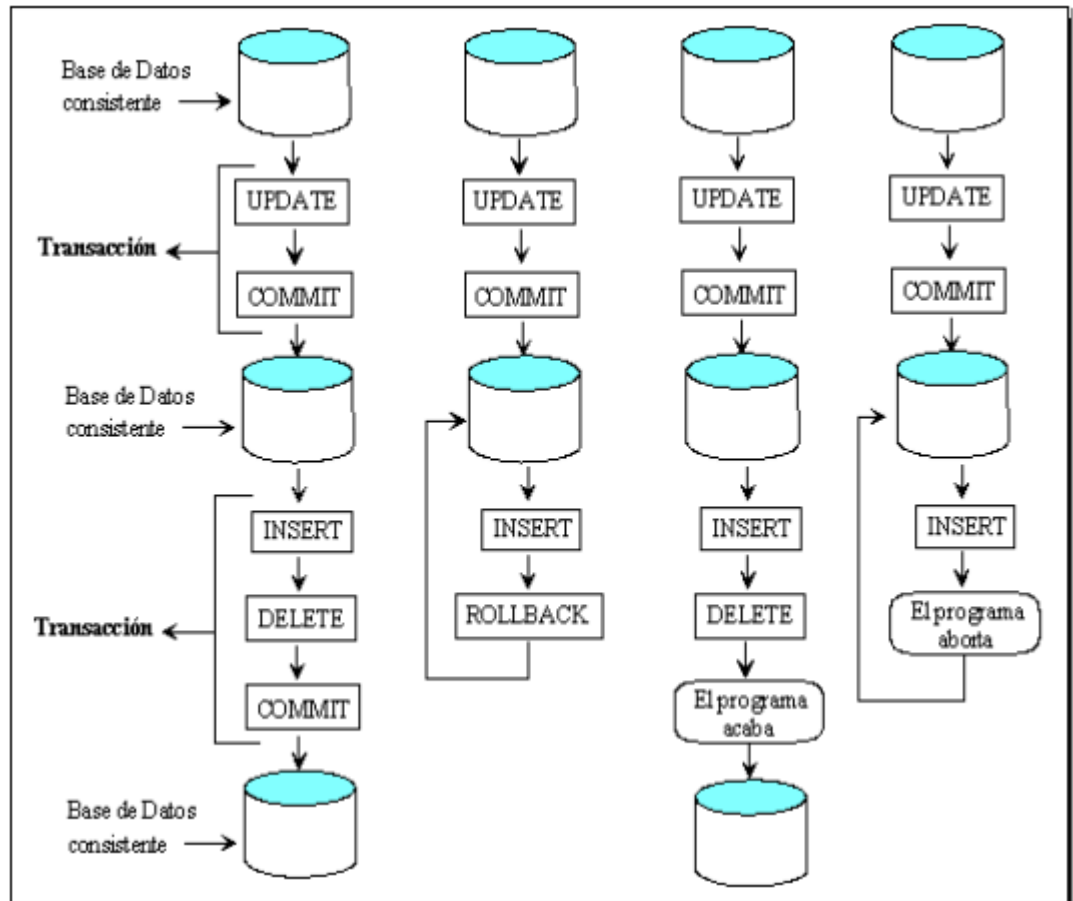
Transacciones

Una transacción es una secuencia de una o más sentencias SQL que juntas forman una unidad de trabajo para el DBMS, el cuál asume que todas las sentencias deben completarse para asegurar la consistencia e integridad de la BD



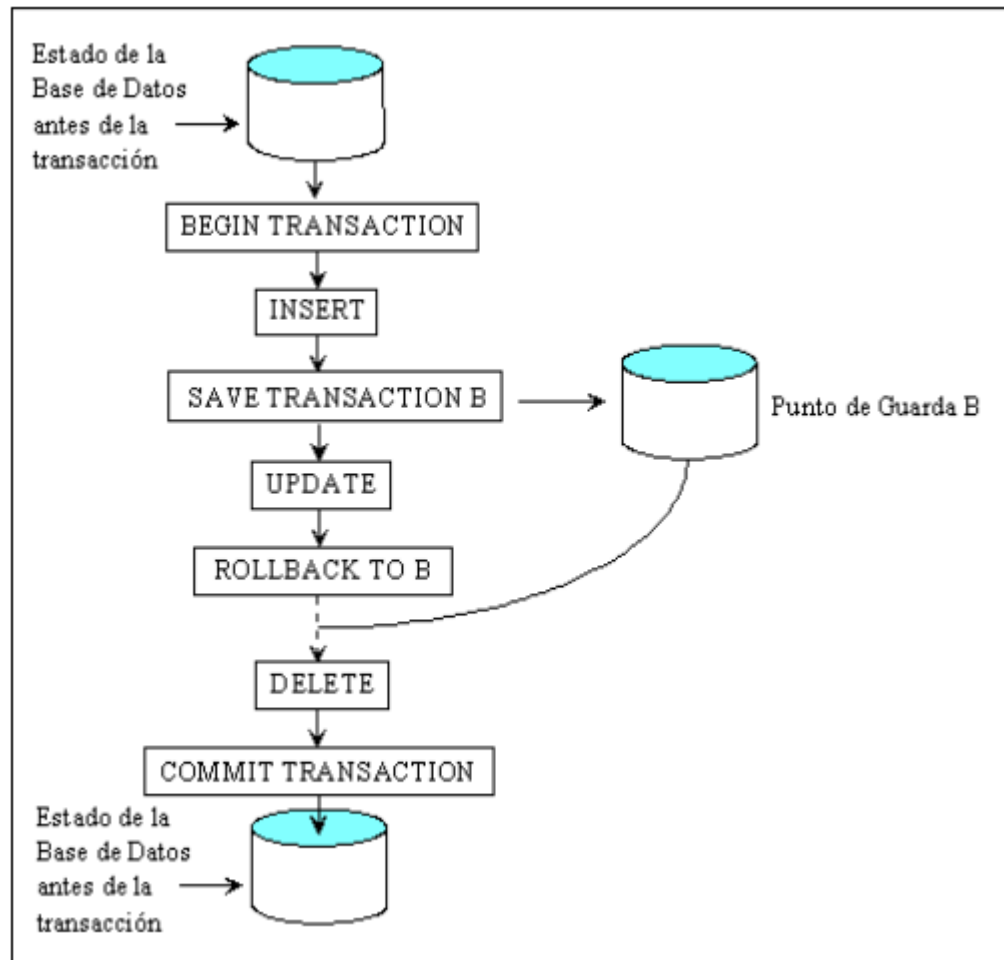
Estándar

SQL estándar soporta las transacciones de base de datos mediante las sentencias COMMIT y ROLLBACK



No estándar

En SQL interactivo se asume que cada sentencia en una transacción Sybase extiende el modelo de transacciones del estándar SQL incluyendo otras sentencias para el procesamiento de transacciones



Procesamiento multiusuario

Cuando dos o más usuarios acceden o ejecutan transacciones concurrentemente, se pueden presentar básicamente tres problemas:

1. Actualización perdida: cuando dos aplicaciones leen datos de la BD, los utilizan para un cálculo y luego los actualizan. Ambas transacciones leen el valor antes de que lo cambie la otra
2. Datos no cumplimentados: cuando una aplicación lee actualizaciones no cumplimentadas de otra y actúa en base a los datos leídos
3. Datos inconsistentes: cuando durante una consulta otra aplicación cumplimenta transacciones que afectan a los datos de la consulta

Niveles de cerramiento

El cerramiento puede ser implementado a distintos niveles de la base de datos

Los cuales pueden ser:

- Cerramiento a nivel de la base de datos
- Cerramiento a nivel de tabla
- Cerramiento a nivel de fila
- Cerramiento a nivel de columnas

Esquemas de cerramiento

Para aumentar el acceso concurrente a una BD, se utilizan esquemas con más de un tipo de cierre, siendo habitual en tal sentido implementar al menos dos tipos de cierre: compartido y exclusivo

T r a n s a c c i ó n A		Transacción B		
	Acción	No Cerrado	Cierre Compartido	Cierre Exclusivo
	No Cerrado	SI	SI	SI
	Cierre Compartido	SI	SI	NO
	Cierre Exclusivo	SI	NO	NO

SQL como middleware

¿Cómo resuelve cada cliente el hecho de acceder a datos en servidores SQL de múltiples vendedores?

La respuesta es a través del middleware específico de la base de datos (API)

El programa se comunica con el DBMS mediante un conjunto de llamadas de función denominadas interfaz de programa de aplicación (API)

Pasos que realiza:

1. Análisis Sintáctico
2. Valida la sentencia contra el catálogo del sistema
3. Optimiza la sentencia, determinando la mejor forma de ejecutarla
4. Genera un plan de aplicación para la sentencia, esto es una secuencia de los pasos a seguir para su realización
5. Ejecuta el plan de aplicación

El API surge por las características procedurales que se apartan del estándar

Hay dos mecanismos para soportar el uso de SQL desde un lenguaje de programación:

- SQL Embebido (ESQL)
- Call-Level Interface (CLI)

El ESQL se sustenta en el estándar ISO ya que no existen características procedurales

En cuanto al CLI hay distintas alternativas, una de las cuales es ODBC (Open DataBase Connectivity) de Microsoft, el cual es una API estándar multivendedor

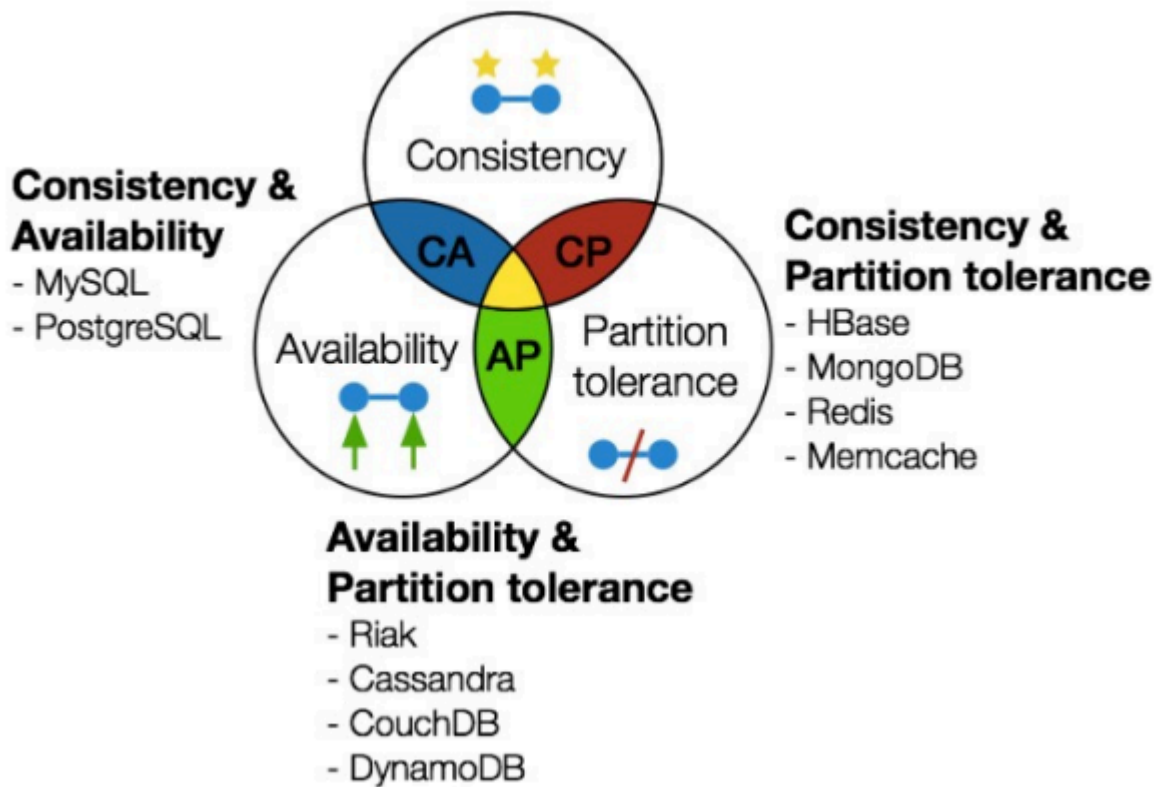
JDBC, permite la ejecución de operaciones sobre bases de datos desde JAVA independiente del SO y del motor de BD

Open Client

Es el software propietario (de cada BD) que garantiza la conectividad entre cliente y servidor

En definitiva dependerá de las aplicaciones cuál de ellos usara y a su vez, la aplicación se verá condicionada por el API que utilice la herramienta de desarrollo

Teorema CAP



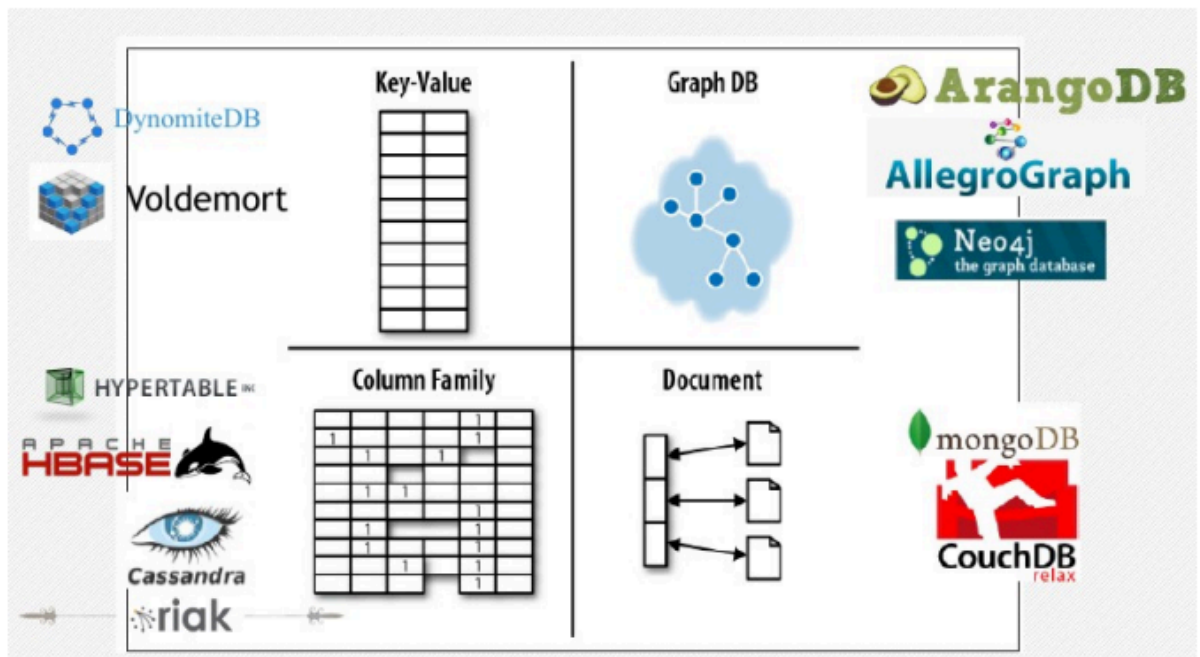
ACID vs BASE

Basically Available (disponible básicamente): está operativo la mayoría del tiempo

Soft state (estado débil): los datos en las diferentes réplicas no tienen que ser mutuamente consistentes en todo momento

Eventually consistent (eventualmente consistente): se asegura la consistencia solo después de que pase cierto tiempo

Taxonomía BBDDs NoSQL



Familia de columnas

La información se obtiene a partir de una clave única

Se tiene una sola tabla enorme y convenientemente indexada

Ejemplos:

- Cassandra (Apache) <http://cassandra.apache.org/>
- BigTable (Google) tablas multidimensionales, celdas con versionado
- HBase (Hadoop)

BBDD documentales

Almacenamiento de documentos, uno o más pares de campo/valor (datos simples o listas)

Cada documento tiene un id

El acceso a los datos es mediante vistas

Posee una arquitectura distribuida con replicación bidireccional y operación off-line

Ejemplos:

- Lotus Domino (IBM)
- CouchDB (Apache)
- MongoDB

MongoDB

- Orientada a documentos

- Almacenados en BSON (binario de JSON)

- Documentos pertenecen a una colección

- Los documentos de una misma colección no están obligados a tener el mismo esquema

- Consultas pasando objetos JSON a través de consola JavaScript

- Drivers para ser utilizada desde Java, PHP, Python, Ruby, etc.

- No existen las transacciones

- No existen los joins

- Las consultas de agregación no son sencillas aunque hay un framework

- Índices de tipo B-Tree

BBDD orientadas a grafos

- Grafos dirigidos

- Almacenan relaciones entre los nodos, preponderancia por sobre los nodos (particularmente útil en redes sociales)

- Ofrecen una API para consultas que recorren los grafos

Clase 8 - Orquestación y coreografía BPEL

Un error común es que una SOA es una nueva versión de servicios web

La distinción entre servicios SOA y servicios web se encuentra en sus respectivos diseños:

- SOA define un modelo para la ejecución de un determinado proceso
- Los servicios web facilitan la aplicación táctica del modelo SOA. Son sólo una de las muchas maneras en las que una SOA puede aplicarse

Impacto de SOA en la infraestructura IT

El beneficio de SOA es que las aplicaciones pueden ser construidas por la combinación de servicios interoperables y estos servicios pueden reutilizarse

Esto conlleva los siguientes riesgos en una infraestructura de IT:

- Tiempos de respuesta malos para los usuarios y los procesos de negocio
- Pérdida de niveles de servicio para funciones críticas del negocio
- El incumplimiento con la industria y las regulaciones gubernamentales
- Infracciones de seguridad
- Insuficiente gestión de los servicios
- Desafíos para el testing

Mientras que históricamente la infraestructura de IT constituyó el apoyo a las aplicaciones, con SOA, su atención se enfoca hacia la gestión de los servicios que prestan apoyo a los procesos de negocio y, por tanto, los resultados del negocio

ESB

En las aplicaciones tradicionales, todos los pasos para completar una transacción están dentro de la aplicación con una simple conexión punto a punto

Con SOA, la aplicación de procesos de negocio inicia la transacción y el ESB provee la mensajería, transformación de datos y el enrutamiento inteligente

El ESB cumple las siguientes funciones:

- Ruteo de mensajes entre servicios
- Conversión de protocolos de transporte entre solicitante y servicio
- Transformación de formatos de mensajes entre solicitante y servicio

- Manejo de los eventos de negocio desde fuentes diversas

Servidor de procesos

El ESB es dirigido por el servidor de los procesos de negocio donde sus procesos de negocio están definidos y gestionados

El servidor de procesos gestiona y orquesta las solicitudes a través del ESB para servicios independientes y poder así completar los procesos de negocio

Los flujos de procesos de negocio son utilizados por el servidor de proceso para representar las interdependencias entre aplicaciones

Orquestación y coreografía de los procesos de negocio

Los procesos de negocio atraviesan la estructura organizativa y definen sus reglas independientemente del proceso

Los servicios resuelven funcionalidades concretas requeridas dentro de cada unidad organizativa y se componen para realizar los procesos de negocio a través de su orquestación y coreografía

Orquestación vs coreografía

	Orquestación	Coreografía
Objetivo	Componer servicios para cumplir con un proceso de negocio dentro de una organización	Componer servicios para colaboración entre organizaciones
Modelo	Jerárquico. Pregunta-Respuesta	Peer-to-peer
Enfoque	Componer servicios y el orden en que son ejecutados para alcanzar el objetivo de un proceso de	Definir la manera en que múltiples partes colaboran para conformar una transacción de negocio

	negocio	
Fundamento	Constituye un servicio en sí mismo	Define la interacción del negocio

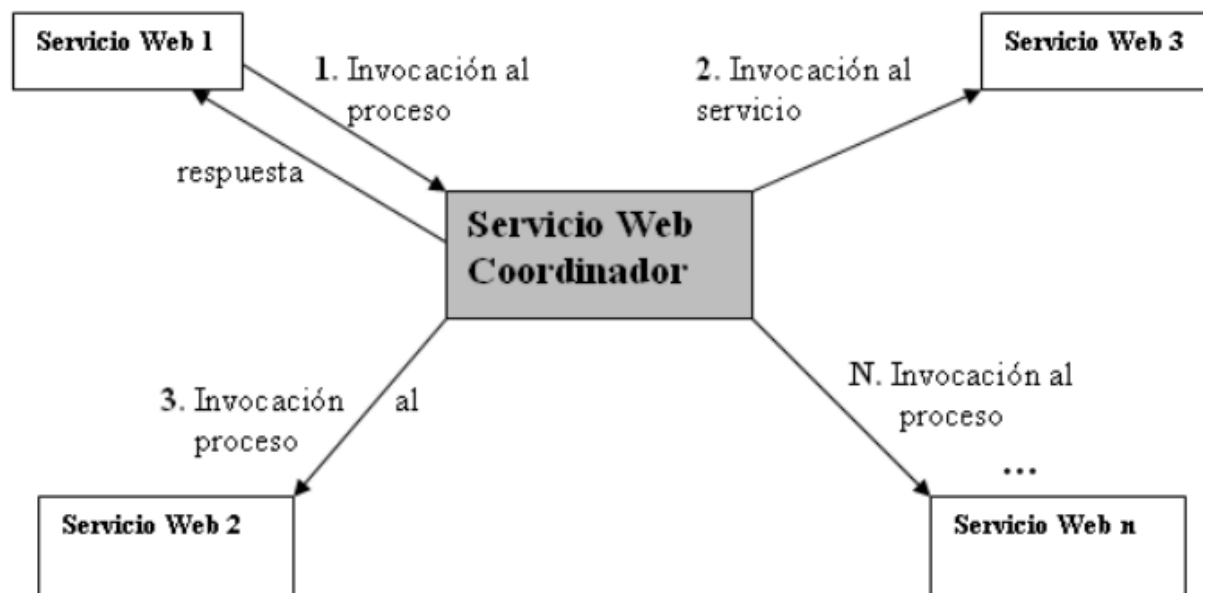
BPEL

BPEL es un lenguaje basado en XML que soporta la tecnología utilizada en los Servicios Web, incluyendo protocolos SOAP, WSDL, UDDI, entre otros

Un proceso BPEL determinará el orden de invocación que se llevará a cabo con los Servicios Web brindando operaciones básicas de condición, iteración y asignación de variables como cualquier lenguaje de programación pero con una orientación hacia el control de procesos de negocios

BPEL4People es una extensión que considera interacción humana

Orquestación de servicios web con BPEL



En la orquestación un proceso central tomará el control de los Servicios Web para luego coordinar la ejecución de las operaciones expuestas por los mismos

Secciones de un documento BPEL

En un documento BPEL existen cuatro secciones principales:

- `<variables>`: define las variables utilizadas por el proceso, en base a un esquema de tipos y mantienen el estado del proceso a lo largo del intercambio de mensajes
- `<partnerLinks>`:
 - Define las diferentes partes que interactúan con el proceso de negocios
 - Cada elemento partner link tiene asociado un partner link type y un nombre de rol
 - Esta información identifica la funcionalidad que debe ser provista por el proceso de negocios e implementada por el servicio para que la interacción sea exitosa
- `<partnerLinkType>`:
 - Permiten describir el rol de los Servicios Web clientes y el proceso mismo involucrados dentro del proceso BPEL
 - Un partnerLink debe tener al menos un partnerLinkType y como máximo dos
- `<faultHandlers>`: contiene los manejadores de excepciones que se ejecutarán en caso de que ocurra una excepción en la invocación a un servicio

Tipos de procesos BPEL

BPEL soporta dos tipos diferentes de procesos de negocios:

- Procesos ejecutables:
 - Los cuales nos permiten especificar detalles exactos del proceso de negocios
 - Estos procesos pueden ser ejecutados por un motor de ejecución
- Protocolos de negocios abstractos:
 - En los cuales únicamente se define el intercambio de mensajes entre las partes
 - Aquí los procesos no incluyen detalles internos de procesamiento y tampoco son ejecutables

Composición de procesos BPEL

La estructura de un proceso de negocios en BPEL consta de pasos, donde cada uno de estos pasos es conocido como una actividad

Las actividades de un proceso se clasifican en:

- Primitivas:
 - Invocación de Servicios Web
 - Espera de solicitudes desde un cliente
 - Generación de una respuesta sincrónica
 - Manipulación de datos
 - Declaración de errores y excepciones
 - Esperar por una cantidad de tiempo
 - Terminar el proceso de ejecución
- Estructuradas:
 - Las actividades estructuradas permiten la combinación de actividades primitivas
 - En este grupo se encuentran:
 - Ejecución secuencial (<sequence>), nos permite declarar un conjunto de actividades las cuales se ejecutarán de forma secuencial
 - Ejecución paralela (<flow>) igual al anterior solamente que las invocaciones se realizarán en paralelo
 - Selector múltiple (<switch>) implementa condicional sobre la siguiente actividad a ejecutar
 - Iterador (<while>) permite iterar sobre un conjunto de actividades
 - Selección de ejecución (<pick>) permite seleccionar un camino de ejecución
 - Fallas (<throw>) permite lanzar excepciones
 - Manejador de excepciones (<faulthandler>) permite tratar las excepciones que se producen
 - Determinar alcance (<scope>) permite delimitar el alcance de un grupo de actividades
 - Terminar (<terminate>) permite finalizar la ejecución de un proceso de negocio

Diferencias entre BPEL y los lenguajes de programación

- BPEL es portable entre plataformas:
 - Un proceso BPEL puede ser ejecutado sobre diferentes motores de ejecución lo cual es muy importante en el desarrollo B2B
- BPEL está especialmente diseñado para el desarrollo de procesos:
 - Esto implica que hará eficiente la ejecución de procesos de larga duración (ya sea minutos, horas, días)
- BPEL también soporta una característica de “deshacer” actividades previas del proceso de negocio:
 - Es posible definir manejadores para estos casos indicando el alcance de los mismos dentro del proceso de ejecución
 - Estas características son conocidas como Long-Running Transactions (LRT)

Exposición e invocación de procesos BPEL

Los procesos BPEL se exponen a los clientes como WS, por lo tanto requieren que se defina su WSDL

Para comenzar el proceso, el cliente deberá invocar una operación del mismo que normalmente será una sola

Cada invocación genera una nueva instancia

Procesos BPEL sincrónicos y asincrónicos

El proceso BPEL puede ejecutarse en forma sincrónica (el cliente espera la respuesta) o asincrónica (el cliente continúa trabajando mientras llega la respuesta invocando a ese cliente)

La diferencia entre ambos:

- Como se retornan los resultados:
 - En ambos casos se esperan en la actividad <receive> o <pick>:
 - En el síncrono se retorna con <reply> y en el asíncrono se usa <invoke> para invocar al cliente y retorna al port type
- Como se declara la interfaz:
 - En las operaciones síncronas se especifican por mensajes <input> y <output> mientras que las asíncronas requieren definición de dos port types, uno para la invocación inicial y otro para la respuesta

```

<bpel:import namespace="http://service.romandk.com/" location=
"http://127.0.0.1:8080/Servicio-Web-C?wsdl" importType="http://schemas.xmlsoap.org/wsdl/"
></bpel:import>
<bpel:import namespace="http://service.romandk.com/" location=
"http://127.0.0.1:8080/StoredProcedureService?wsdl" importType="http://schemas.xmlsoap.org/wsdl/"
></bpel:import>
<bpel:import namespace="http://servicio.romandk.com/" location=
"http://127.0.0.1:8080/InformeTributario?wsdl" importType="http://schemas.xmlsoap.org/wsdl/"
></bpel:import>
<bpel:import namespace="http://servicio.romandk.com/" location=
"http://127.0.0.1:8080/InformeTerritorial?wsdl" importType="http://schemas.xmlsoap.org/wsdl/"
></bpel:import>
<bpel:import location="orquestacionArtifacts.wsdl" namespace="http://eclipse.org/bpel/sample"
importType="http://schemas.xmlsoap.org/wsdl/" />

```

import

Lista todos los documentos wsdl de los servicios web utilizados en el proceso BPEL

```

<bpel:partnerLinks>
  <bpel:partnerLink name="client" partnerLinkType="tns:orquestacion" myRole=
"orquestacionProvider"/>
  <bpel:partnerLink name="territorial" partnerLinkType="tns:territorialPLT" myRole=
"territorialServer" partnerRole="territorialClient"></bpel:partnerLink>
  <bpel:partnerLink name="tributario" partnerLinkType="tns:tributarioPLT" myRole=
"tributarioServer" partnerRole="tributarioClient"></bpel:partnerLink>
  <bpel:partnerLink name="caratulacion" partnerLinkType="tns:caratularPLT" myRole=
"caratularServer" partnerRole="caratularClient"></bpel:partnerLink>
  <bpel:partnerLink name="categoria" partnerLinkType="tns:categoriaPLT" myRole=
"categoriaServer" partnerRole="categoriaClient"></bpel:partnerLink>
</bpel:partnerLinks>

```

PartnerLink

Lista los servicios participantes en el flujo del proceso de negocio, los roles que cumplen y partnerLinkType asociados

```

<bpel:variables>
  <bpel:variable name="input" messageType="tns:orquestacionRequestMessage" />
  <bpel:variable name="output" messageType="tns:orquestacionResponseMessage" />
  <bpel:variable name="territorialIn" messageType=
"ns1:ObtencionInformeTerritorial_getInformeTerritorial" />
  <bpel:variable name="territorialOut" messageType=
"ns1:ObtencionInformeTerritorial_getInformeTerritorialResponse" />
  <bpel:variable name="tributarioIn" messageType=
"ns1:InformeTributarioService_obtenerInformeTributario" />
  <bpel:variable name="tributarioOut" messageType=
"ns1:InformeTributarioService_obtenerInformeTributarioResponse" />
  <bpel:variable name="deudasize" type="ns2:int" />
  <bpel:variable name="indexdeuda" type="ns2:int" />
  <bpel:variable name="caratulaIn" messageType=
"ns3:StoredProcedureCaratular_caratularTramite" />
  <bpel:variable name="caratulaOut" messageType=
"ns3:StoredProcedureCaratular_caratularTramiteResponse" />
  <bpel:variable name="categoriaIn" messageType="ns3:InvocaCC_invocaservicioCC" />
  <bpel:variable name="categoriaOut" messageType="ns3:InvocaCC_invocaservicioCCResponse" />
</bpel:variables>

```

Variables

Sirven para mantener el estado de los proceso de negocio. permiten almacenar mensajes que mantendrán los valores recibidos y enviados por los servicios. Los tipos de cada variable pueden ser mensajes WSDL, un tipo de dato simple definido en XML o un elemento XML.

```

<bpel:sequence name="main">
  <bpel:receive name="receiveInput" partnerLink="client" portType="tns:orquestacion"
    operation="process" variable="input" createInstance="yes" />
  <bpel:assign validate="no" name="requerimiento">
    <bpel:copy>
      <bpel:from>
        <bpel:literal xml:space="preserve"><tns:PruebaTramiteResponse xmlns:tns="http://eclipse.org/bpel/tramite"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <tns:result></tns:result>
</tns:PruebaTramiteResponse>
</bpel:literal>
      </bpel:from>
      <bpel:to variable="output" part="payload"></bpel:to>
    </bpel:copy>
    <bpel:copy>
      <bpel:from part="payload" variable="input">
        <bpel:query queryLanguage="urn:oasis:names:tc:wsbpel:2.0:sublang:xpath1.0">
          <![CDATA[tns:input/tns:arg0]]>
        </bpel:query>
      </bpel:from>
      <bpel:to part="payload" variable="output">
        <bpel:query queryLanguage="urn:oasis:names:tc:wsbpel:2.0:sublang:xpath1.0">
          <![CDATA[tns:result/tns:arg0]]>
        </bpel:query>
      </bpel:to>
    </bpel:copy>
  </bpel:assign>
  <bpel:if name="Estado viabilidad ?">
    <bpel:condition><![CDATA[output.payload/tns:result/tns:arg0/tns:tramite/tns:estado = 2]]></bpel:condition>
    <bpel:sequence name="viabilidad legal"><bpel:assign validate="no" name="Assign">
      <bpel:else>
    </bpel:if>
    <bpel:reply name="replyOutput" partnerLink="client" portType="tns:orquestacion"
      operation="process" variable="output" />
  </bpel:sequence>

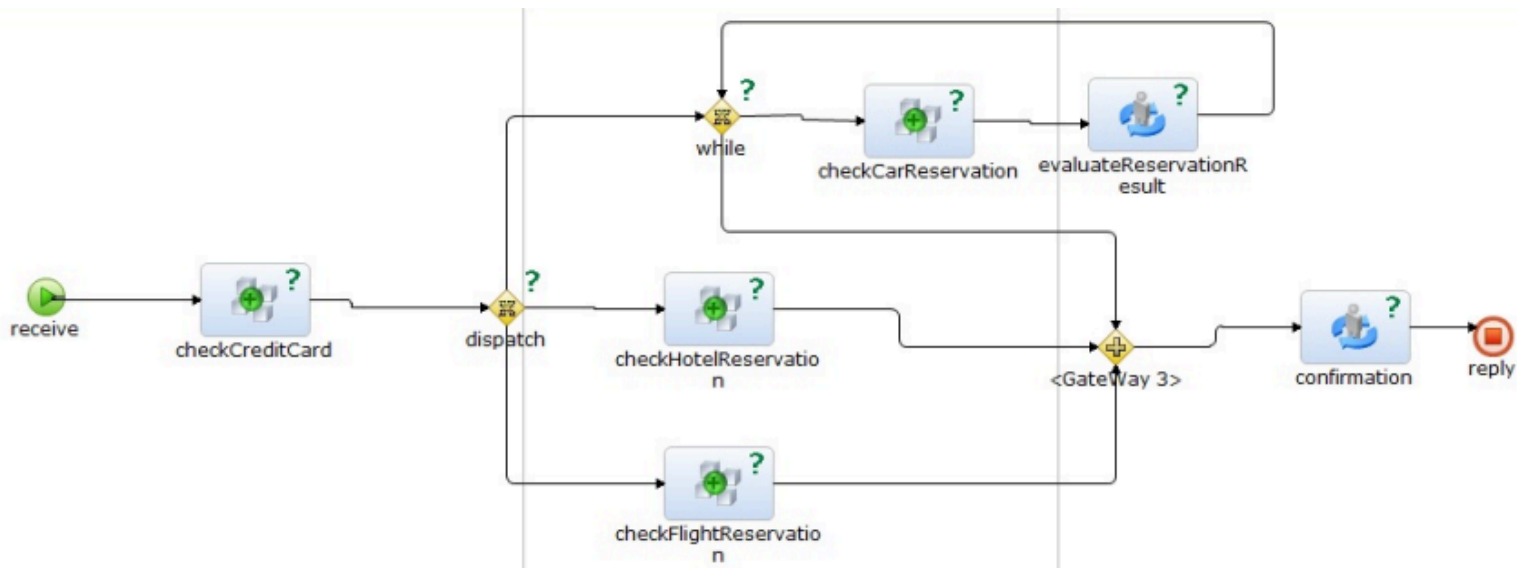
```

Lógica del proceso

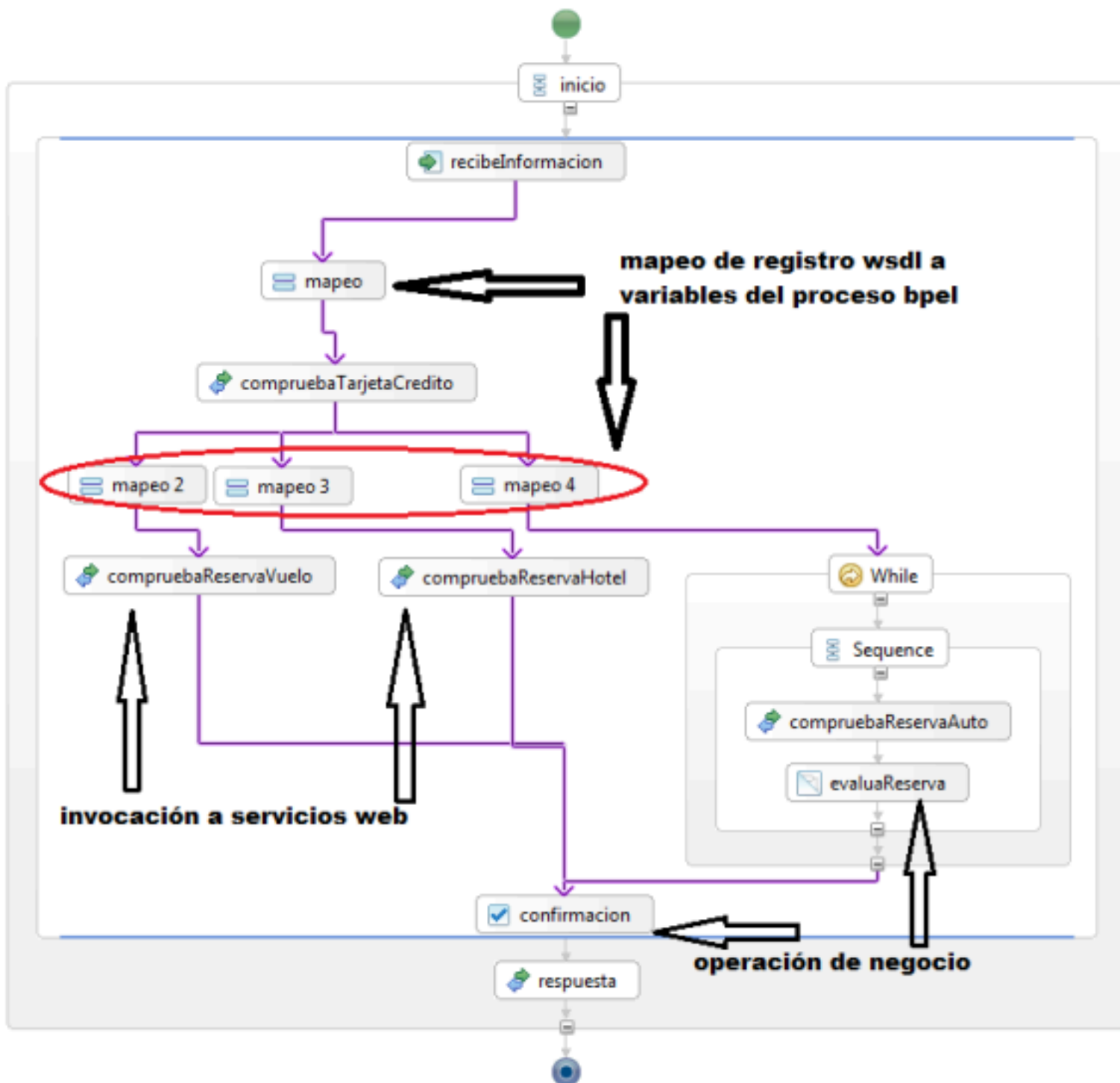
Dentro del tag **<bpel:sequence>** se define la lógica de proceso de negocios, y la declaración de los receive, invoke, assign, ... de los procesos de negocios

24

Proceso de reserva de viaje en BPMN



Proceso de reserva de viaje en BPEL



Clase 9 - Ecosistemas digitales de integrabilidad

Transformación digital hacia el ciudadano

La UX del ciudadano es global con todas las organizaciones involucradas en su evento de vida

El camino hacia el gobierno inteligente



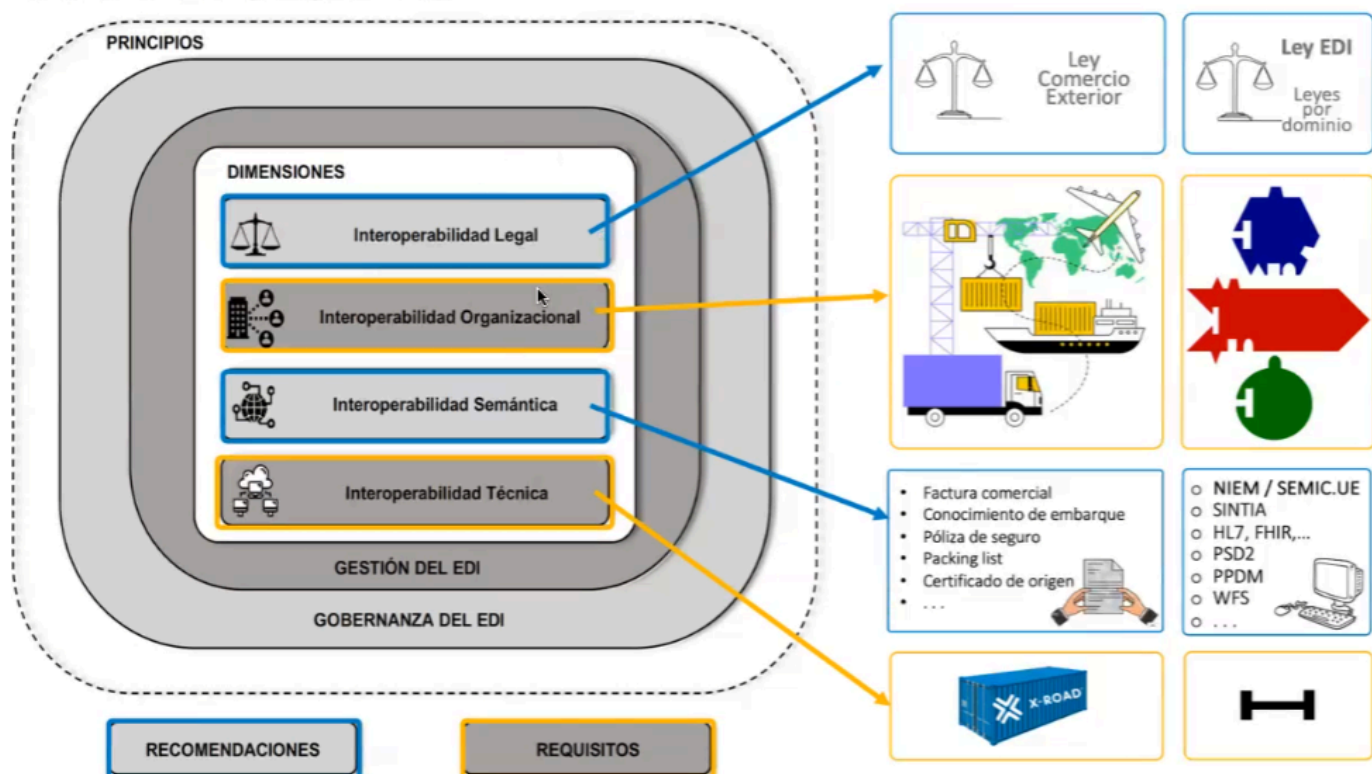
Principios que guían hacia el gobierno inteligente

- Una sola vez ("Once only"):
 - Si alguna organización del estado me pidió un dato alguna vez, ninguna otra organización del estado me lo puede volver a pedir
 - Interoperabilidad de fuentes auténticas
- Procesos proactivos ("push"):
 - Si se sabe que luego de un trámite hay que hacer otro, el organismo que termina el primer trámite puede iniciar por mí el segundo trámite
 - Coordinación de eventos de vida
- Interfaz abierta:

- Posibilidad de extender funcionalidades de una aplicación simplemente porque al lado de la interfaz humana hay apis que nos permite extender los servicios
- Integración de las aplicaciones
- Una ventanilla unificada

Normas

IRAM 17610-1 EDI



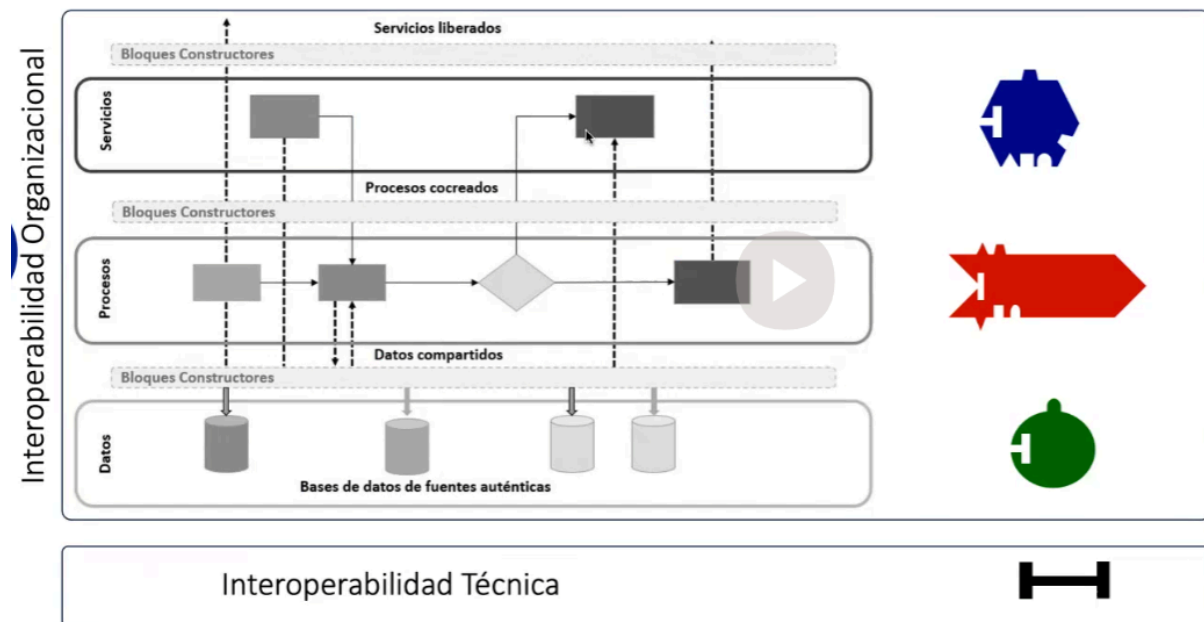
Necesita una gobernanza y una gestión

Analogía con la logística (1er punto logística, 2do punto EDI)

- Interoperabilidad técnica:
 - Definir un container estándar para que todo entre ahí adentro y que se pueda transportar por grúas estándar sin necesidad de intervención humana
 - En el mundo digital es la caja digital que creó Estonia que tiene todos los condimentos de seguridad, inviolabilidad, trazabilidad, de identidad digital para que los sistemas puedan comunicarse directamente entre sí

- Interoperabilidad semántica:
 - Documentos físicos o digitales: facturas, pólizas, listados, certificados, etc.
 - Estándares digitales para poder comunicarse entre sistemas sin la interacción humana, donde el container (seguro) debe ser el mismo para todos
- Interoperabilidad organizacional:
 - Forma de transporte
 - Fichitas que encastran según datos (verde), procesos (rojo) y servicios (azul)
- Interoperabilidad legal:
 - Ley de comercio exterior
 - Ley EDI + leyes por dominio

IRAM 17610

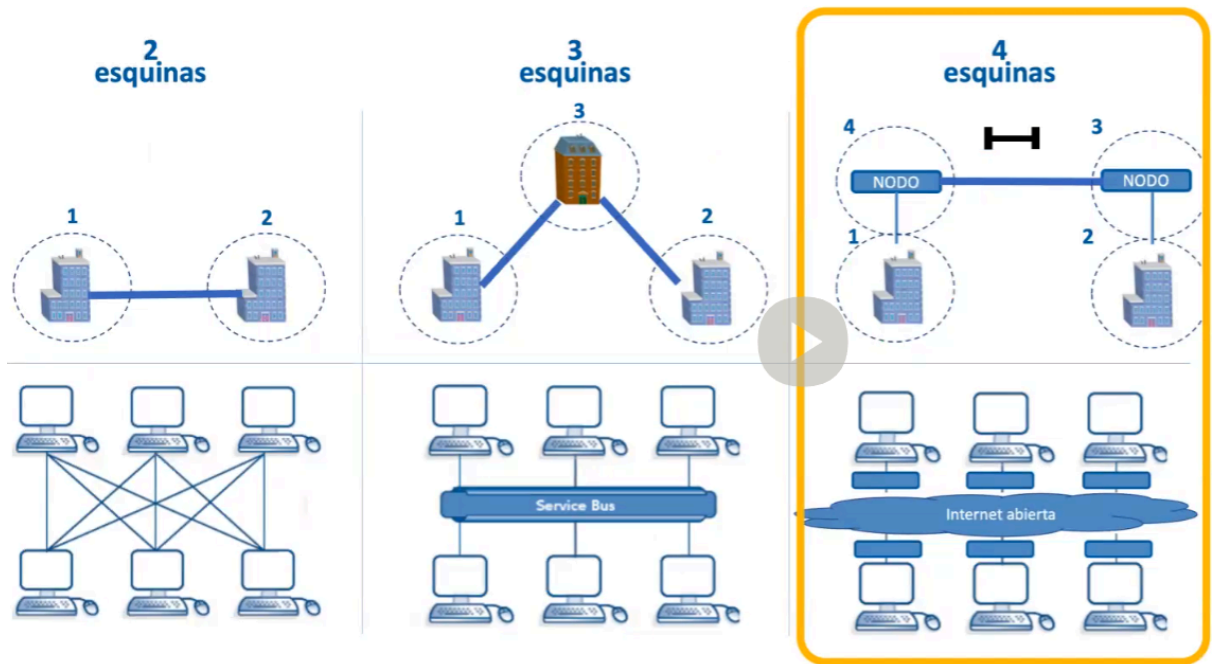


La norma nos permite es:

- Tener seguridad (interoperabilidad técnica)
- Poder hacer que cualquier fuente de datos pueda ser usado por cualquier proceso interorganizacional (interoperabilidad entre datos y procesos)
- Poder hacer que cualquier proceso interorganizacional con cualquier fuente de datos pueda ser usado por cualquier aplicación (servicio) (interoperabilidad entre datos/procesos y servicios)

Todo esto teniendo las cuestiones legales y de confidencialidad que correspondan

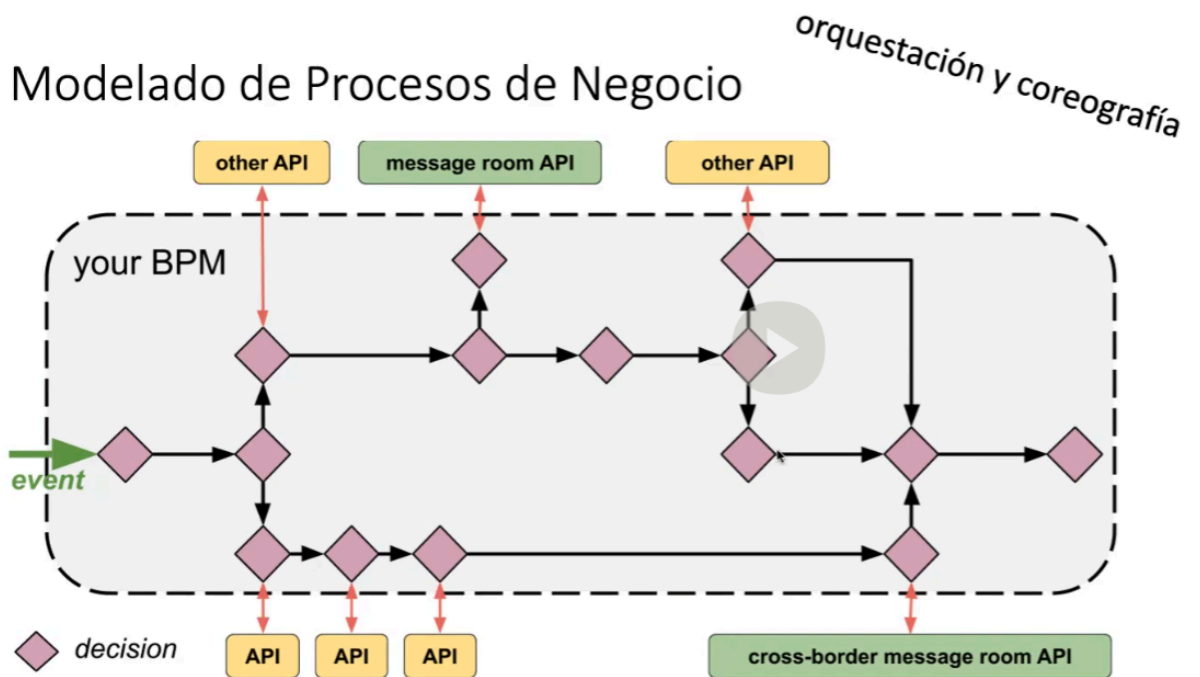
Modelos de interoperabilidad



- 2 esquinas:
 - Yo se español y quiero hablar con un chino, o el chino aprende español, o yo aprendo chino
 - Uno debe adaptarse al otro
 - Si luego aparece un alemán, ahora tenemos 2 problemas, el alemán conmigo y con el chino
 - El crecimiento es exponencial
 - Es una arquitectura estrella, todos conectados con todos
- 3 esquinas:
 - Tener un traductor o políglota
 - Nunca tengo la certeza de que al chino le esté llegando lo que yo quise decir (hay una delegación en una tercera parte)
 - Hay un único punto de falla, si se cae, cagamos
 - Es una arquitectura con un bus de servicio, cada uno con el bus
- 4 esquinas:
 - Lengua universal

- Todos hacen un esfuerzo para aprender inglés y pueden hablar entre todos
- Es una arquitectura donde la comunicación es mediante la internet abierta

Modelado de procesos de negocio



Real time economy

Real time economy (RTE): Protocolos proactivos /Smart Contracts



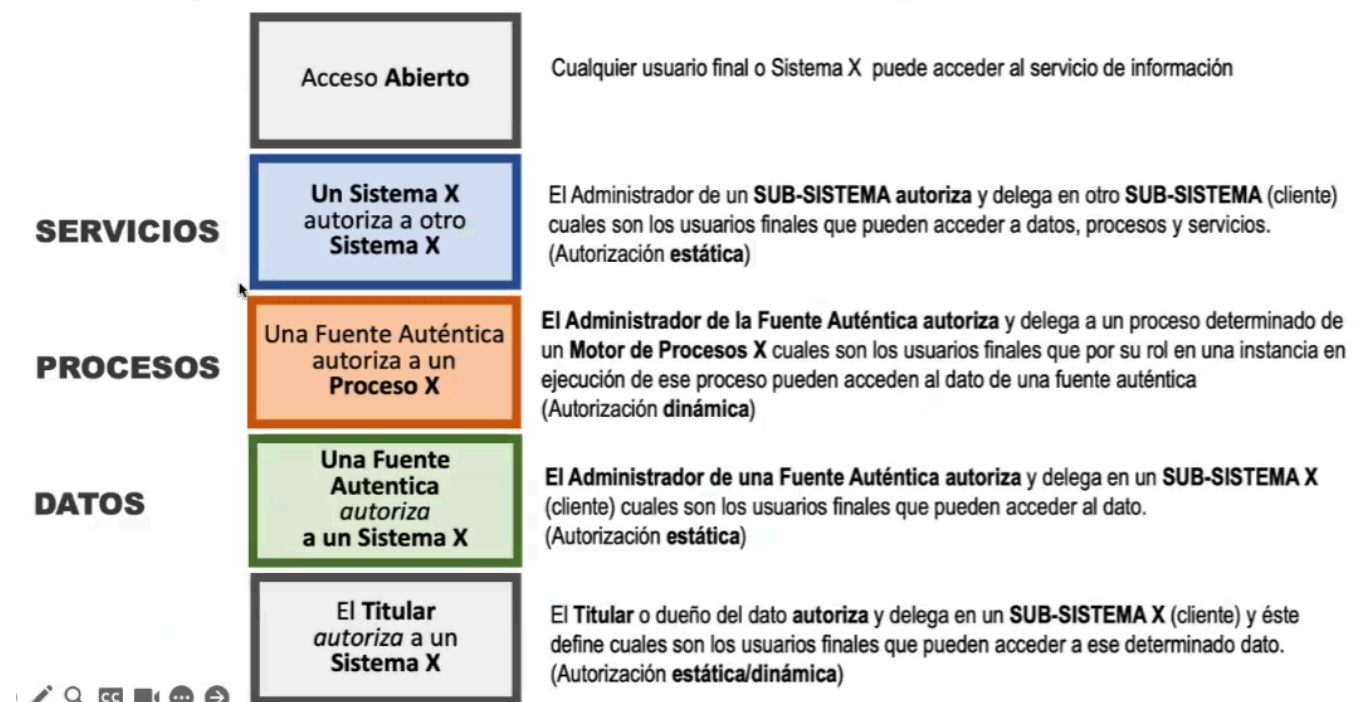
La velocidad a la que suceden las operaciones sin interacción humana es muchísimo más elevada

Capas del modelo organizacional

El otro ingrediente interesante que agregan las capas de la dimensión organizacional es que quien produce un dato, decide o autoriza quién puede usarlo: las organizaciones autorizan a una aplicación a acceder a los datos, y las aplicaciones autorizan a los usuarios (yo como productora de alcohol autorizo a X bar a venderlo, y el bar autoriza a cada borracho)

Esto se evalúa a partir de auditorías

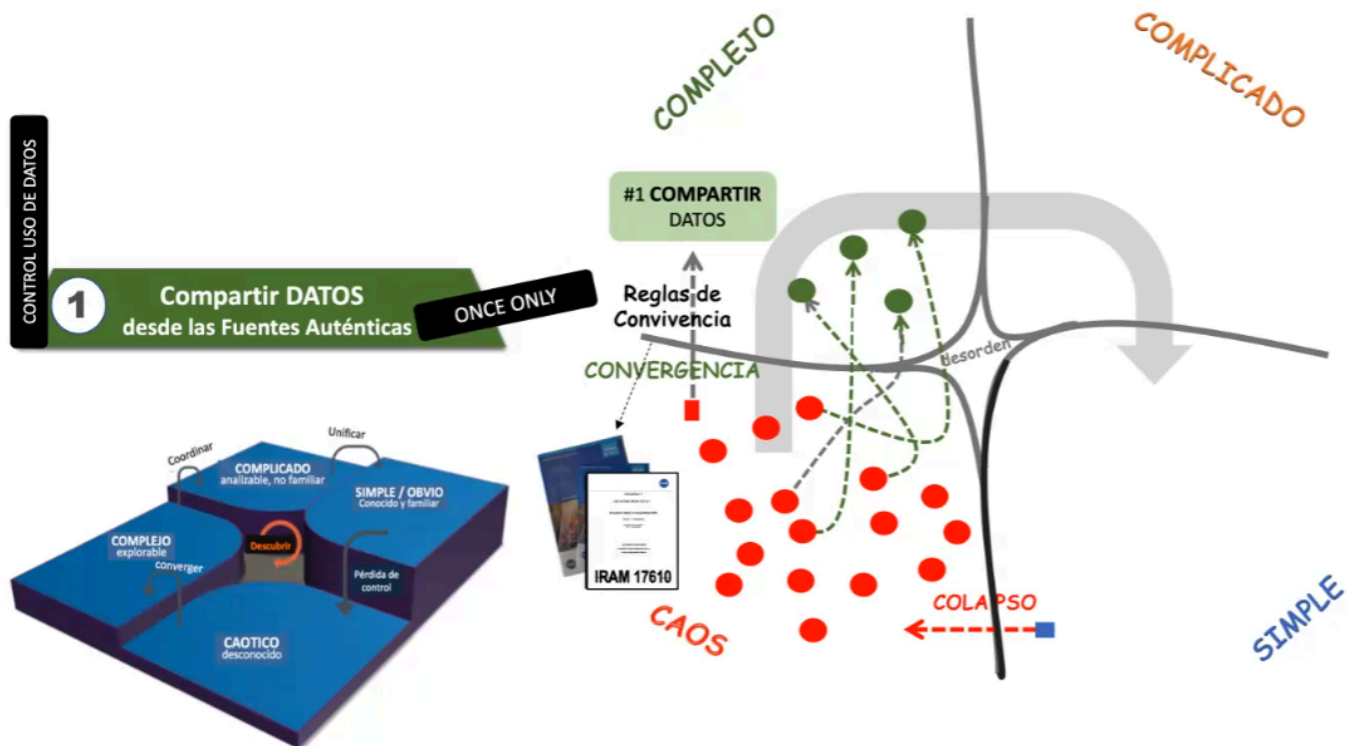
Esquemas de autorización desde los Datos personales a Datos abiertos



Ruta de transformación hacia un ecosistema digital inteligente



¿Cómo se hace?



Se debe hacer de manera colectiva

Se debe salir del estado caótico

2 reglas principales de ciberseguridad:

- Segmentar la red: ya está hecho
- Zero trust: ya está hecho, cada vez que se quiere comunicar con alguien se debe autenticar

Se deben seguir los siguientes pasos:

1. Compartir datos:

- A través de la convergencia
- Pasar de un estado caótico y desconocido a uno complejo y explorable

2. Coordinarse:

- Pasar de un estado complejo y explorable a un estado complicado y no familiar, pero analizable

3. Unificar:

- Pasar de un estado complicado y no familiar, pero analizable a un estado simple/obvio, conocido y familiar

4. Tener cuidado de no perder el control y colapsar al caos

Estos pasos se dan mediante el descubrimiento

Clase de repaso

Evolución de los sistemas distribuidos

- [Aplicación Monolítica](#)
- [Arquitectura Distribuida \(2 capas y N capas\)](#)
- [Arquitecturas Distribuidas con objetos \(estándar CORBA\)](#)
- [Integración de aplicaciones](#)
- [Arquitecturas Orientadas a Servicios](#)
- [Procesos de negocio](#)

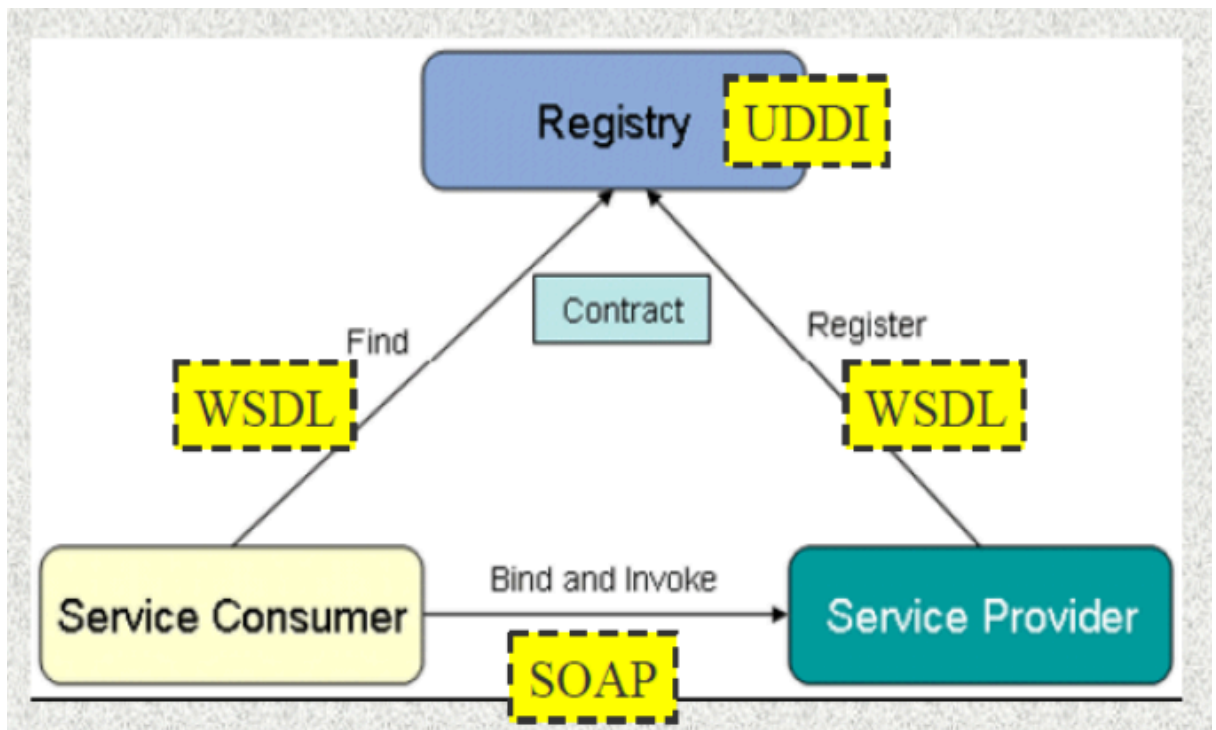
La idea de middleware

Existen dos grandes clases de middleware:

- General:
 - Transporte
 - Red
- Específico del servicio:
 - RPC
 - SQL
 - Web Services
 - MOM

Servicios: un middleware posible

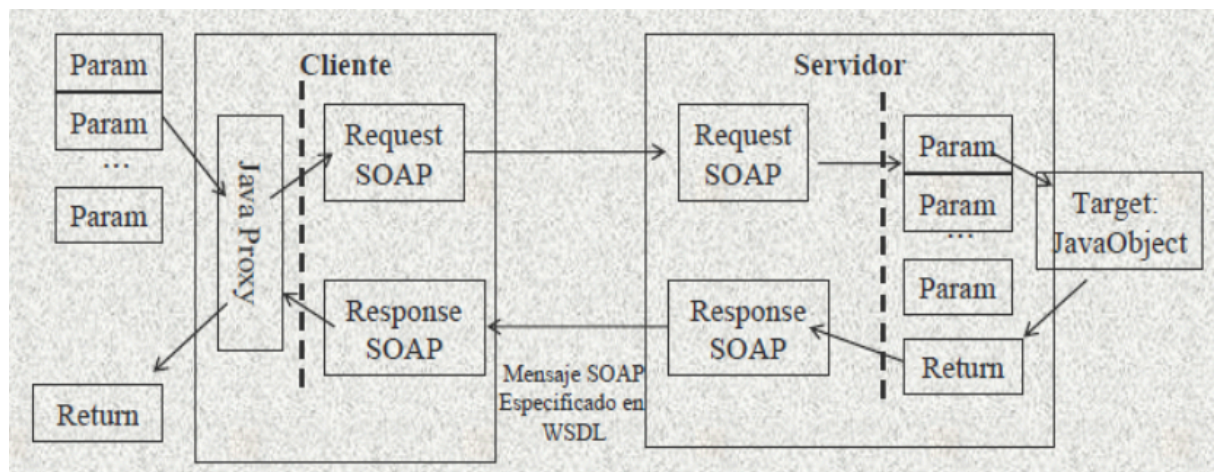
Web services



REST y SOAP

	REST	SOAP
Formato del mensaje	XML	XML dentro de SOAP
Definición del interfaz	No es necesario	WSDL
Transporte	HTTP	HTTP, JMS, FTP, etc.

SOAP en Java

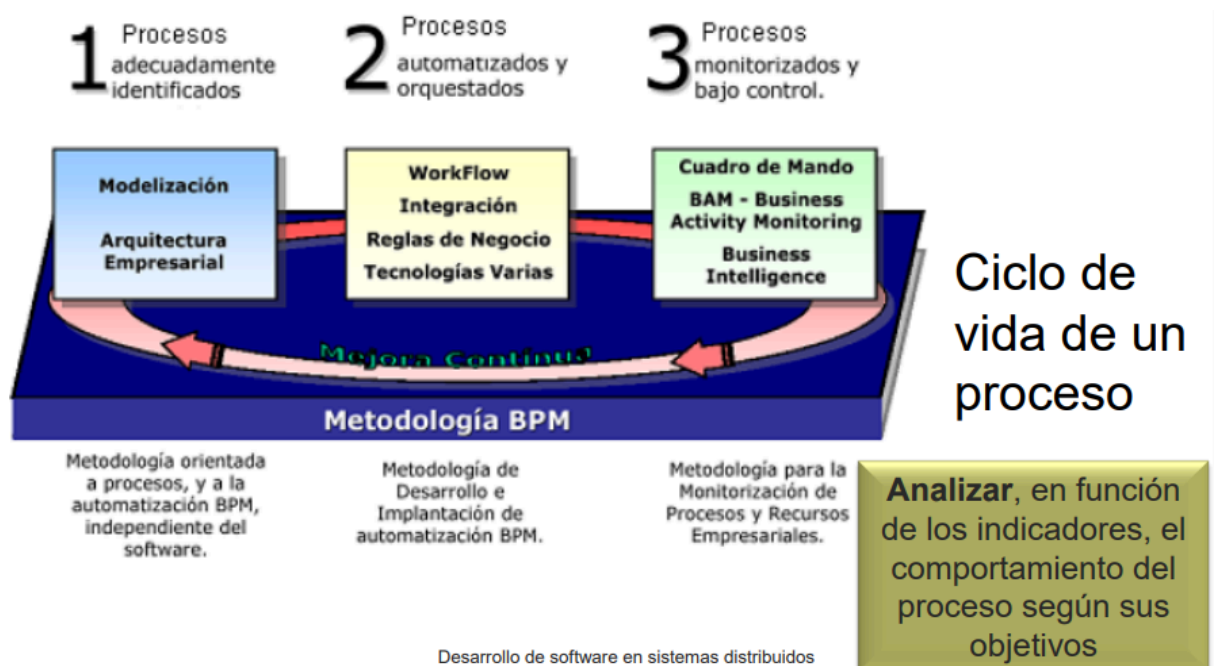


Los procesos de negocio como orquestadores de servicios

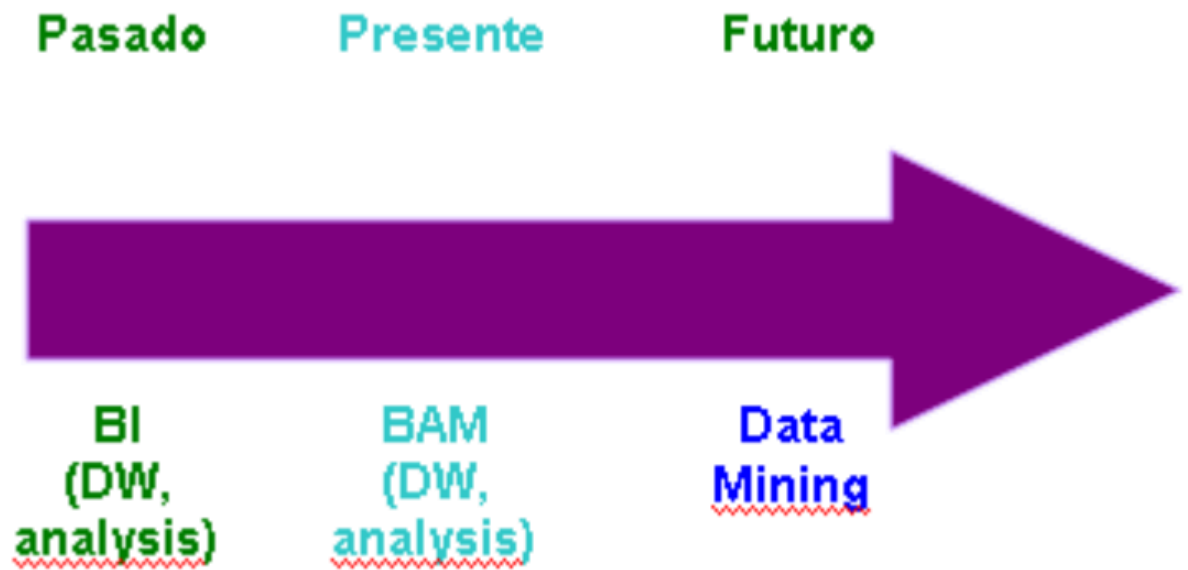
Un proceso de negocio es un conjunto de actividades que se realizan en coordinación en un ambiente organizacional y técnico

Es la definición, instancia de ejecución y archivo de un proceso de negocio

Etapas de trabajo con la metodología BPM



BAM vs BI



Ejecución de procesos: BPEL

BPEL es un lenguaje basado en XML que soporta la tecnología utilizada en los Servicios Web, incluyendo protocolos SOAP, WSDL, UDDI, entre otros

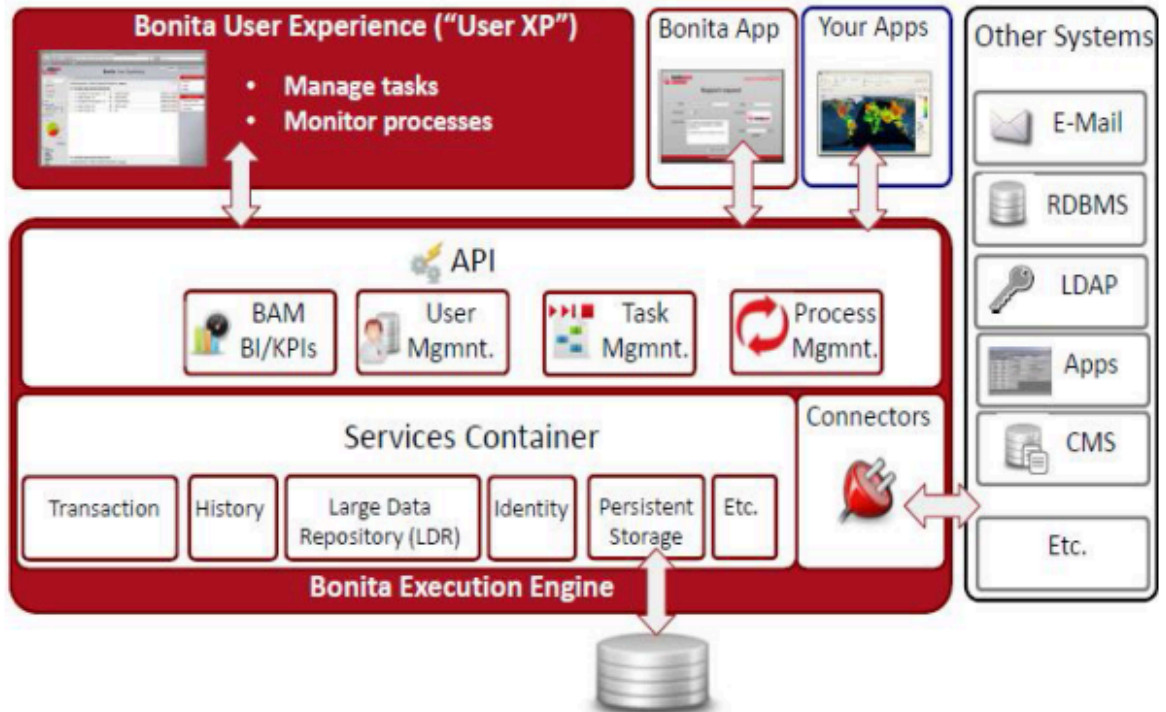
ESB y motor de procesos

El ESB es dirigido por el servidor de los procesos de negocio donde sus procesos de negocio están definidos y gestionados

El servidor de procesos gestiona y orquesta las solicitudes a través del ESB para servicios independientes y poder así completar los procesos de negocio

Los flujos de procesos de negocio son utilizados por el servidor de proceso para representar las interdependencias entre aplicaciones

BonitaSoft Run-Time Architecture



Servidores SQL y NoSQL

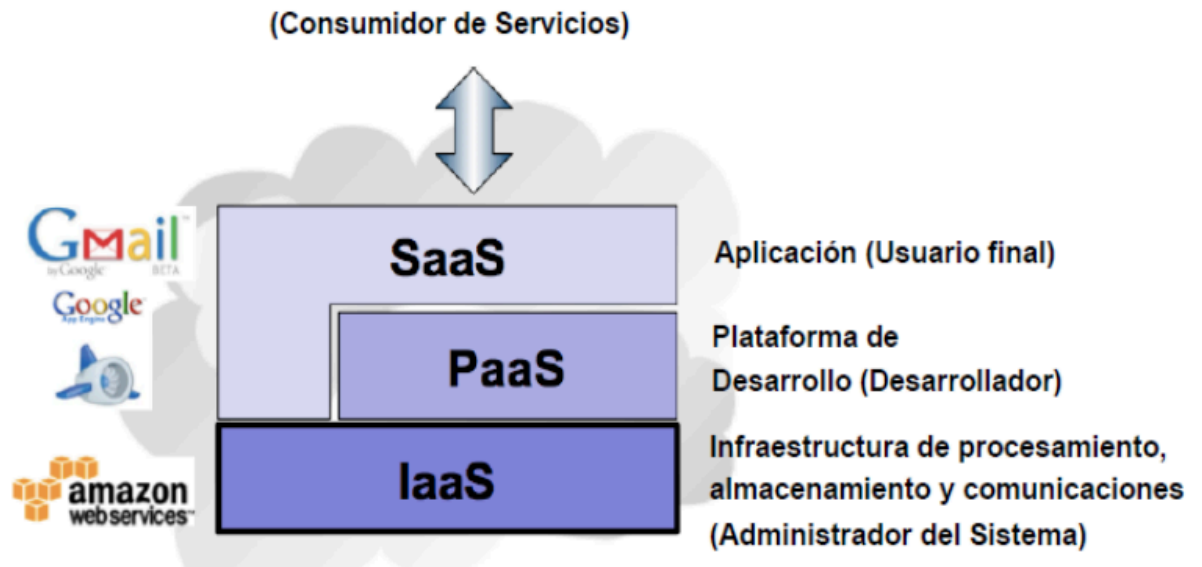
SQL vs NoSQL

SQL	NoSQL
Permite combinar tablas y relacionar información	No lo permite o muy limitado
Facilita distribuir Bases de Datos Relacionadas	Distribuir grandes cantidades de información
Gestión de datos y sus relaciones	No existen estos conceptos
Complicado el escalado Horizontal	Escalado horizontal

Servidores SQL

- Clasificación por procesamiento
- Clasificación por almacenamiento
- Control de concurrencia
- Cómo y cuándo se resuelve el SQL

Modelo de computación en la nube



Seguridad en la nube

- Amenazas y desafíos a nivel de red, de host y de aplicaciones
- La seguridad en cloud públicos
- A nivel de red y de host, ningún desafío de seguridad es intrínseco del modelo de servicio
- A nivel de aplicaciones, las amenazas provienen de las aplicaciones Web

Ecosistemas Digitales de Integrabilidad

Un ecosistema digital es un tipo generalizado de entorno informático ubicuo compuesto por especies ubicuas, geográficamente dispersas y heterogéneas (Dong et. al, 2011) y los servicios publicados por estas especies reflejan las mismas características

Un EDI es una comunidad de organizaciones miembros de un mismo ecosistema que:

1. Respetar mínimas reglas de convivencia digital
2. Aplica estándares y componentes de software para poder utilizar y reutilizar los servicios comunes del ecosistema (Ugalde et al., 2024a)

Un ecosistema digital de integrabilidad será exitoso siempre que aborde realmente necesidades de las personas y agregue valor

